



Efficient Gradient-Based Learning of LPV Models with Linear Fractional Representation

Master Thesis, 45 EC

**Department of Electrical Engineering
Control Systems
Master Systems & Control**

Author: R. Drenth - 1395734

Supervisor: Prof. Dr. Ir. Roland Tóth
Co-Supervisor: Dr. Ir. Maarten Schoukens

This report was made in accordance with the TU/e Code of Scientific Conduct
for the Master thesis

Eindhoven, September 19, 2025



Abstract

This thesis addresses the surrogate modelling of nonlinear systems by *Linear Parameter Varying* (LPV) models with *Linear Fractional Representation* (LFR) and rational scheduling dependency. A novel parameterization is developed to constrain LPV-LFR models to be well-posed. The parameterization is used for gradient-based joint estimation of LPV models and scheduling maps under complex noise scenarios. Furthermore, the reliability of the estimation approach is improved by the introduction of an initialization and bootstrapping approach based on the best linear approximation and the use of unconstrained multiple shooting. These approaches are validated by Monte Carlo studies on datasets from two different non-linear models. Finally, a model augmentation framework is introduced to allow the incorporation of prior system knowledge into the identification process. Together, the findings in this thesis provide a reliable and computationally efficient method for the joint-estimation of parsimonious LPV models with rational scheduling dependency through the use of a novel well-posed parameterization.

Contents

1	Introduction	1
1.1	Background	2
1.2	Research questions	3
1.3	Outline	4
1.4	Notation	4
2	Modelling LPV systems in the LFR framework	5
2.1	Introduction	5
2.2	Well-posed LPV-LFR models	8
2.3	Identification Setting	9
2.4	Model Structures	9
2.5	Identification Method	11
2.6	Parameter Initialization	12
2.7	Simulation Example	13
2.8	Discussion	14
3	Multiple-Shooting-Based Identification	16
3.1	Introduction	16
3.2	Multiple Shooting-based optimization	16
3.3	Simulation Example - CMG	17
3.4	Simulation Example - NL-MSD	18
3.5	Discussion	20
4	Bootstrapping approaches for the identification of LPV models	23
4.1	Introduction	23
4.2	BLA-based bootstrapping procedure	23
4.3	Simulation Example - CMG	25
4.4	Simulation Example - MSD	25
4.5	Discussion	27
5	Model Augmentation of Linear Parameter-Varying Models	29
5.1	Introduction	29
5.2	Model Augmentation in LPV-LFR identification setting	29
5.3	Bootstrapping procedure for augmented models	30
5.4	Simulation Examples	31
5.5	Discussion	34
6	Conclusions & Recommendations	36
6.1	Conclusions	36
6.2	Recommendations	36
7	References	38
A	Appendix A	41
A.1	Figures associated with Chapter 3	41
A.2	Figures associated with Chapter 4	41

1 | Introduction

As the demands on performance of systems in engineering grow, the ubiquitous *Linear Time-Invariant* (LTI) framework shows its limits. High performant, model-based controllers rely, as the name implies, on accurate models of systems to be controlled. Modelling the inherently nonlinear (NL) behaviour present in many physical systems, via LTI methods, results in inaccurate models. When the aim is to push performance to its limits, these inaccurate models can result in limitations on the achievable performance under control. These limitations are becoming more apparent as demands on performance of controlled systems increase, requiring more accurate models.

As an alternative to the LTI framework we consider the *Linear Parameter-Varying* (LPV) framework. LPV models retain the linear structure of LTI models, but with the notable difference that its parameters can now vary as a function of a *scheduling variable*. Through the variation of the scheduling variable, LPV models are capable of representing *Time-Varying* (TV) as well as NL behaviour, with a simple linear structure. This structure provides a good trade-off between the simple, but restricted modelling capabilities of the LTI framework and more complex NL or TV methods. Additionally, this linear structure enables the application of sophisticated controller synthesis tools within the LPV framework. (see, e.g., [1]). Notably, similar to the LTI case, these controllers can be synthesized with performance guarantees. The LPV framework also enjoys the support of various software toolboxes, implementing modeling and control tools ([2], [3]).

The subject of LPV data-driven modelling, i.e., system identification is still an actively researched topic. Currently, the majority of LPV identification research has focused on methods with (partial) prior knowledge of the scheduling. Considered are internally scheduled models, where direct measurement of the scheduling variable (see Figure 1.1a) is available and externally scheduled models, where the *scheduling map*, which maps signals to the scheduling variable, is known and its inputs are measured. (see Figure 1.1b). Within the setting with known scheduling, a wide range of estimation methods exists, often extensions of LTI identification methods. We provide an overview of some notable methods. Using *Input-Output* (IO) structures, PEM methods similar to the LTI case have been developed for LPV-IO models, including representation of various noise models (*AutoRegressive model with eXogenous input* (ARX) to *Box-Jenkins* (BJ)) [4]. As controller synthesis methods typically rely on LPV *State-Space* (SS) models, significant research effort has been put into the estimation of LPV-SS models under (partially) known scheduling. Like in the IO case, PEM methods for LPV-SS models have been researched, including identification of plant and noise models [5]. Additionally, significant research on the topic of *Subspace Identification* for LPV-SS models has been performed ([6], [7]).

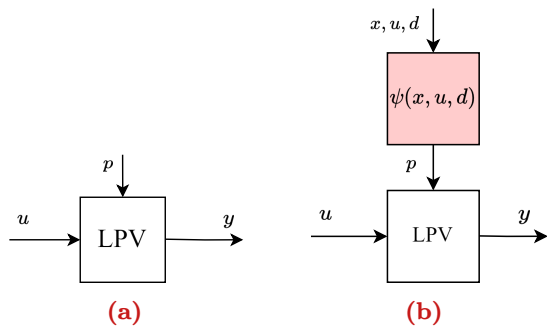


Figure 1.1: Estimation with (partially) known scheduling. Direct access to the scheduling variable shown in (a). Known scheduling map based on measurable signals shown in (b).

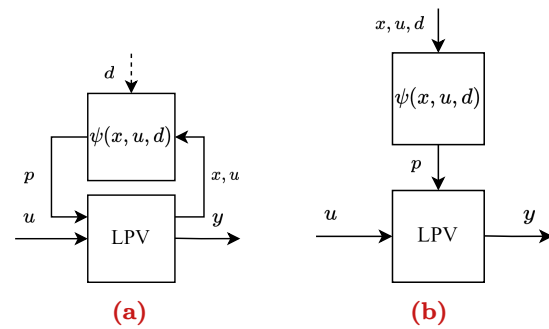


Figure 1.2: Joint-estimation of LPV model and scheduling map, scheduling map and LPV model both unknown. Self-scheduled setting shown in (a), externally scheduled setting shown in (b).

The previously outlined identification methods rely on (partial) knowledge of the scheduling, this knowledge is however not always available. Instead, recent research efforts have focused on joint-estimation methods (see Figure 1.2) in which both LPV model and scheduling map are identified simultaneously. This approach allows the estimation of LPV models solely from IO data, or possibly with the inclusion of measurable exogenous signals known to affect scheduling. In [4], a joint estimation approach is performed

for externally scheduled LPV-IO models. To identify the scheduling map a sparsity-based basis pursuit approach is used to identify the scheduling map from a large set of basis functions. The first deep-learning based approach to joint estimation is proposed in [8]. Using autoencoders an LPV model is identified. However, the scheduling map and matrix functions are not separable, resulting in a model unsuitable for control purposes. Subsequently, in [9] an alternative approach is proposed where *Feedforward Neural Networks* (FNNs) are used to learn the scheduling map and an LPV-SS model in a self-scheduled setting. In this approach a firm split between input and state dependent scheduling is enforced. Additionally, the forced normalization in this method could result in numerically ill-conditioned models. In [10] a different encoder-based approach to joint-estimation is taken. This approach supports both self-scheduled and externally scheduled cases. The method estimates LPV-SS models with innovation noise model under affine scheduling dependency. The method has shown great estimation performance on examples studied, but at significant computational cost. For this purpose, a truncated prediction loss function is used such that the problem can be solved in parallel. This requires the use of state-estimates from the state-encoder network. Initially in [11] and further explored in [12], a computationally efficient self-scheduled joint estimation approach is proposed. Here the scheduling map is parametrized as a FNN and the LPV model as LPV-SS model with affine scheduling dependency. Optimization is performed using auto-differentiation according to a PEM cost. The use of a novel optimization approach results in significantly shorter training times than in [10]. However, some unresolved downsides to this method exist. The use of a traditional (untruncated) prediction loss results in a computational problem unsuited for parallelization. Additionally, high sensitivity to initial parametrization is found, resulting in the need for multiple optimization runs. Furthermore, it should be noted that the current joint-estimation research is restricted to LPV-IO and LPV-SS representations, no research has been performed on direct joint-estimation of LPV-LFR models and scheduling map.

The above mentioned black-box methods have been shown to provide accurate results in modeling of NL systems, however prior system knowledge, for example based on physical laws through *First Principle Modeling* (FPM), is left unused. On the other hand, purely FPM based or *mechanistic* models often perform worse than well-identified black-box models, for example due to unmodelled dynamics. Instead, *model augmentation* methods for NL systems (sometimes also called *hybrid modeling* methods in literature) have been proposed [13]. In this framework, mechanistic models are augmented with black-box terms, typically neural networks, to estimate unmodelled behaviour. Often, parameters of the mechanistic part of the model are updated during identification. This provides a number of key benefits: (i) Better initialization as prior knowledge does not have to be learned by black box models; (ii) Improved extrapolation to unseen data as NL black-box models often struggle generalizing to input-state combinations that have not been incorporated in training data, (partially) mechanistic models can improve this behaviour; (iii) Better interpretability of models as, after identification, the mechanistic part of the model retains the predetermined, physics inspired relations. As a result, the updated model parameters can retain physical meaning. To the best of the authors knowledge, there is no prior work found in the literature regarding integration of model augmentation methods into an LPV framework, despite its potential benefits over general NL methods.

1.1 | Background

LPV models are capable of representing NL behaviour through the concept of *LPV embedding*. In this process, an LPV model capable of representing all solutions of the original NL system is found. To achieve this, a latent variable p (the scheduling variable) is introduced. The NL model (Figure 1.3a) can be reformulated by defining a functional dependency of the plant on p and characterizing p such that in combination with the functional dependency, the reformulation is equivalent to the original NL system

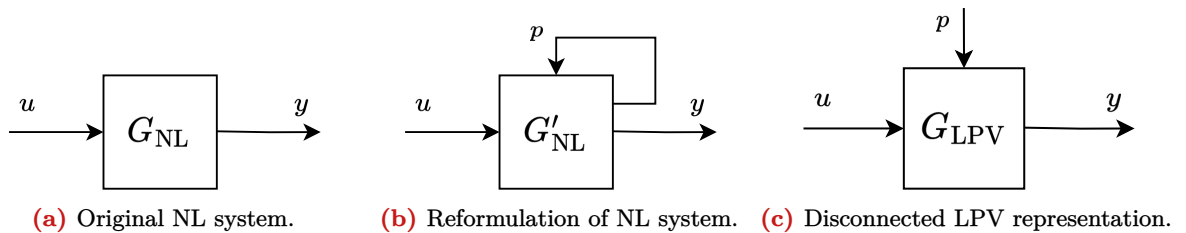


Figure 1.3: LPV embedding process.

(Figure 1.3b). If the internal connection of p is severed, and p is instead assumed a free, but known variable, then the remaining input-output relations are linear. By choosing p to be free, but bounded to the same values as achievable in the connected scenario, the behaviour of the NL representation is embedded in the LPV representation. (Figure 1.3c) However, this freedom of p may introduce solution trajectories not present in the original NL behaviour. This difference between the trajectories admitted by the NL system and the LPV system is typically referred to as overbounding or conservativeness. When the LPV embedding is subsequently used in controller synthesis, the additional behaviour must be addressed, possibly leading to conservativeness in the controller. A major challenge in modelling of LPV systems is finding models that minimize overbounding. At the same time, the total amount of scheduling variables should be kept low, to reduce computational challenges in controller synthesis and keep the amount of model parameters low.

As mentioned, in practice, LPV controller synthesis methods often rely on LPV-SS models, specifically with an affine dependency on the scheduling variables. However, in case the underlying dependency is NL rather than affine, this approach can lead to an increase in the required amount of scheduling variables, resulting in a significant growth in model parameters. Additionally, the resulting models can lead to unnecessary conservatism during controller synthesis. A possible solution to this problem is the use of LPV models with a *Linear Fractional Representation* (LFR) model structure. This model structure admits rational dependency structures which can reduce the effects of overbounding, yet is still suited for LPV controller synthesis (see e.g., [14] for a comparison). However, so far, little research on the identification of LPV-LFR models has been performed. In the first paper on LPV-LFR identification, [15], an instrumental variable approach to perform parameter estimation of a SISO LPV-LFR models with a single scheduling variable is used. In [16] and [17], LPV-LFR models with OE noise model are considered, models are optimized using nonlinear optimization to minimize a prediction error cost function. In [18] SISO LPV identification with rational dependency on a single scheduling variable is considered. This paper shows that under certain conditions, the scheduling trajectory can be manipulated to identify a model using convex optimization. In [19] a kernelized canonical correlation analysis approach is used to first estimate a state sequence based on input, output and scheduling data. A nonlinear coordinate descent method is then used to minimize a least squares cost of the state and output prediction error. Unfortunately, all of these methods fail to address a major challenge in LPV-LFR identification: ensuring well-posedness of the models during identification. When LPV-LFR models admit rational scheduling dependencies, identification can result in an ill-posed model in terms of algebraic loops without solutions. By assuming affine dependency in LPV-LFR models the problem of ill-posedness can be avoided, however these types of models are equivalent to affine LPV-SS models, preventing potential benefits in the reduction of overbounding of rational LPV-LFR models. In current research, the well-posedness condition is typically (and naively) assumed to be met, however outside of toy examples this makes identification of rational LPV-LFR models infeasible. To the authors knowledge, no prior method that guarantees well-posedness of LPV-LFR models with rational scheduling dependency exists.

1.2 | Research questions

Based on the overview given in the previous section, three challenges in the field of LPV system identification have been presented. First, as applicability for controller synthesis hinges on parsimonious models, identifying LPV-LFR with rational scheduling dependency is of great value. However, no reliable method for ensuring well-posedness in LPV-LFR models with rational dependency currently exists. To resolve this, the first key goal of the research will be to answer the following research question:

- *How can the well-posedness, during estimation, of LPV-LFR models with rational scheduling dependency be guaranteed?*

Secondly, to enable computationally efficient joint-estimation we aim to extend the gradient-based optimization methods in [12] to LPV-LFR models. Currently however, this method can result in inaccurate surrogate models, typically relying on selecting the best from a set of models with different initialization. Without proven reliability, in terms of converging to accurate models, the usefulness of the proposed identification method is extremely limited. Therefore, there is a strong need to improve the reliability of the methodology. To achieve this we aim to answer the following research question:

- *How can reliable convergence to accurate surrogate models be achieved using gradient-based algorithms for efficient estimation of self-scheduled LPV-LFR models?*

Finally, to ease identification of LPV models based on prior knowledge, such as LTI models or FPM based models, an LPV model augmentation framework is desired. As current model augmentation methods typically result in general NL models there is a need for the development of LPV specific methods. With this in mind, the following research question is formulated:

- *How to incorporate prior information efficiently in the identification of LPV-LFR models?*

By answering these questions and incorporating the results of the noise modelling approach of [20], we aim to provide the theory and tools that: (i) provide reliable and computationally efficient methods for the joint-estimation of parsimonious LPV models and scheduling maps for use in control; (ii) provide guaranteed well-posedness of LPV models with rational scheduling dependency models; (iii) uses a unified LPV-LFR model structure for use of affine and rational dependency models, also suited for model augmentation; (iv) support modelling of scheduling-dependent noise models.

1.3 | Outline

In the following chapters, the research questions detailed above will be discussed and answered. Each chapter after this one will feature a short introduction including an overview of relevant previous research. Additionally each chapter includes a short discussion based on the theory and results obtained. This chapter aims to give the reader an understanding of the problem setting and research question, providing background information where necessary. In Section 2, well-posedness conditions for LPV-LFR models are derived. A description of the identification approach is then given and validated on a simple example. Subsequently, in Section 3, multiple shooting methods are incorporated into the identification setting of Section 2, improving the reliability of convergence of the identification method, as shown on two identification examples. Next, in Section 4, a structured initialization and bootstrapping procedure based on the *Best Linear Approximation* is formulated. This method is shown to significantly aid the nonlinear optimization process used in identification. In Section 5, the identification method and model structure introduced in Section 2 is extended to support model augmentation, allowing the incorporation of prior information into the identification process. Finally, in Section 6, conclusions based on all other chapters are drawn, and recommendations, based on the discussions of each chapter are made.

1.4 | Notation

Let $\mathbb{R}$ and $\mathbb{Z}$ denote the set of real numbers and integers respectively. We define I and $\mathbf{0}$ to be identity and zero matrices of appropriate dimension. Given a matrix $A \in \mathbb{R}^{n \times m}$, the notation $A \succ 0$ ($A \succeq 0$) denotes that the matrix A is positive (semi-) definite, similarly $A \prec 0$ ($A \preceq 0$) denotes that the matrix A is negative (semi-) definite. For non-symmetric matrices, positive (semi-) definiteness and negative (semi-) definiteness are defined for their symmetric component, e.g. $\frac{A+A^\top}{2} \succ 0$. Furthermore, we define $\text{vec}(A)$ to be the nm -dimensional column vector obtained by stacking all columns of A , i.e. $\text{vec}(a_1, \dots, a_m) = [a_1^\top \dots a_m^\top]^\top$. The uniform distribution with lower bound a and upper bound b is denoted as $\mathcal{U}(a, b)$. The normal distribution with mean μ and standard deviation σ is defined as $\mathcal{N}(\mu, \sigma^2)$. When the elements of a matrix are sampled from a distribution, for example $A \sim \mathcal{U}(-1, 1)$, it is assumed that the samples are *identically and independently distributed* (i.i.d.).

2 | Modelling LPV systems in the LFR framework

This chapter discusses the joint-estimation of well-posed LPV-LFR models and scheduling maps under complex noise scenarios. First, background information on the LPV-LFR model structure is given, after which the novel well-posedness conditions and parameterizations are presented. Next, these conditions are integrated in a system identification framework based on the work of [20] to handle complex noise scenarios. Finally, the identification method under well-posedness conditions is demonstrated on a simulation example and compared to affine-dependency LPV-LFR models.

2.1 | Introduction

The Linear Fractional Representation is a general model representation that allows one to model dynamical systems with a wide range of properties, such as nonlinearities and uncertainty, in a unified manner. It originally gained popularity in the field of robust control [21] where it is used to model uncertain systems. For LPV systems it is instead used to incorporate the parameter-varying nature. With an LFR, LPV systems are represented by an interconnection between a nominal system G and its parameter-varying components represented by a matrix function $\Delta(p)$ with linear dependence on the scheduling signal p .

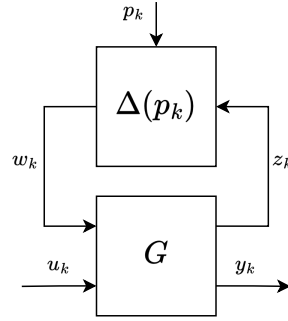


Figure 2.1: Illustration of the interconnection between LPV plant and delta block in LFR representations.

Let us define an LPV-LFR as a pair $(G, \Delta(p))$ where the interconnection in continuous time follows the standard form:

$$G : \begin{cases} \begin{bmatrix} \dot{x}(t) \\ z(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} A_x & B_w & B_u \\ C_z & D_{zw} & D_{zu} \\ C_y & D_{yw} & D_{yu} \end{bmatrix} \begin{bmatrix} x(t) \\ w(t) \\ u(t) \end{bmatrix}, \quad w(t) = \Delta(p(t))z(t), \end{cases} \quad (2.1)$$

where $u(t) \in \mathbb{U} \subseteq \mathbb{R}^{n_u}$ is the input signal, $x(t) \in \mathbb{X} \subseteq \mathbb{R}^{n_x}$ is the state of the system, $w(t), z(t) \in \mathbb{R}^{n_w}$ are the latent variables, and $y(t) \in \mathbb{Y} \subseteq \mathbb{R}^{n_y}$ is the system output and $A_x, \dots, D_{yu}$ are real matrices of appropriate dimensions. $p(t) \in \mathbb{P} \subseteq \mathbb{R}^{n_p}$ is the scheduling variable and $t \in \mathbb{R}$ is continuous time. The matrix function $\Delta(p(t)) : \mathbb{P} \rightarrow \mathbb{R}^{n_w \times n_w}$, commonly referred to as the Delta-block, is defined as:

$$\Delta(p(t)) = \begin{bmatrix} p_1(t)I_{\eta_1} & & 0 \\ & \ddots & \\ 0 & & p_{n_p}(t)I_{\eta_{n_p}} \end{bmatrix}, \quad (2.2)$$

where η is a vector of integers denoting the number of repetitions per scheduling variable and where $I_{\eta_1}, \dots, I_{\eta_{n_p}}$ are identity matrices of appropriate size to realize the correct total dimension of $\Delta(p(t))$. This interconnection is equivalent to a LPV-SS representation with rational dependency on $p(t)$:

$$G_\Delta : \begin{cases} \begin{bmatrix} \dot{x}(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} \mathcal{A}(p(t)) & \mathcal{B}(p(t)) \\ \mathcal{C}(p(t)) & \mathcal{D}(p(t)) \end{bmatrix} \begin{bmatrix} x(t) \\ u(t) \end{bmatrix}, \end{cases} \quad (2.3)$$

where $\mathcal{A}(p(t)), \mathcal{B}(p(t)), \mathcal{C}(p(t)), \mathcal{D}(p(t))$ are now matrix functions of $p(t)$. This is achieved by eliminating the latent variables $w(t)$ and $z(t)$ in (2.1) by substitution of

$$z(t) = (I - D_{zw}\Delta(p(t)))^{-1}(C_z x(t) + D_{zu}u(t)). \quad (2.4)$$

Thus, for this interconnection to be well-posed we require $I - D_{zw}\Delta(p(t))$ to be non-singular for all values of $p(t)$. As this representation admits rational LPV-SS models, it is a generalization of commonly used

affine LPV-SS models. Notably, affine-dependency models are represented by taking (2.1) with $D_{zw} = 0$, resulting in a model set which always satisfies the well-posedness condition. Furthermore, given an LPV-SS model with affine dependency an equivalent LPV-LFR realization can easily be recovered using *Singular Value Decomposition* (SVD) methods [21].

2.1.1 | The trade-off between affine and rational dependency models

In Section 1 we stated that the use of rational dependency LPV-LFR models can reduce overbounding compared to affine dependency models, and subsequently reduce conservatism in controllers. It is important to understand, that in the LPV embedding process, specifically the last step, (see Figure 1.3c) knowledge of the scheduling map, except for its range is discarded. Specifically, any relationship or *inherent coupling* [22] between scheduling variables is discarded. For example, if multiple scheduling variables are dependent on the same state, treating the scheduling variables independently can lead to combinations of scheduling variables that could not occur in the original NL system. By extending the LPV model structure to allow rational dependency structures, rather than capturing inherent couplings in the scheduling map, of which the knowledge is later discarded, they can be captured in the dependency on the scheduling signal itself. Crucially, this coupling is not ignored during controller synthesis. Besides reducing the overbounding, moving complexity from the scheduling map to the dependency structure can also reduce the required number of scheduling variables, further improving usability in controller synthesis.

To illustrate the trade-off made when using affine dependency versus rational dependency in constructing LPV models we consider a simple example. Consider the NL *Mass-Spring-Damper* (MSD) system in Figure 2.2. The model consists of a lumped mass m connected to the fixed world by a NL spring $k(x)$ and a NL damper $d(x)$. The mass is actuated by an input force u and its output is the displacement x . The dynamics of the system are described by the following differential equation:

$$\ddot{x} = \frac{1}{m} (u - k_1 x - k_2 x^3 - d_1 \dot{x} - d_2 \dot{x} \dot{x}), \quad (2.5)$$

We consider two types of LPV embeddings of (2.5). First, we consider an affine dependency-based LPV-embedding; secondly we consider an unrestricted (in this case polynomial) dependency-based embedding. For the model with affine-dependency we introduce the scheduling variables $p_1 = x$, $p_2 = x^2$, capturing position-dependent nonlinearities in the NL spring. This results in the following affine-LPV embedding:

$$\ddot{x} = \frac{1}{m} (u - (k_1 + k_2 p_2)x - (d_1 + d_2 p_1)\dot{x}). \quad (2.6)$$

By taking $x_1 = x$, $x_2 = \dot{x}$, this can be represented in LPV-LFR form by

$$G_{\text{aff}} : \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ z_1 \\ z_2 \\ y \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ -\frac{k_1}{m} & -\frac{d_1}{m} & 1 & 1 & \frac{1}{m} \\ 0 & -\frac{d_2}{m} & 0 & 0 & 0 \\ -\frac{k_2}{m} & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ w_1 \\ w_2 \\ u \end{bmatrix}, \quad \Delta_{\text{aff}}(p) = \begin{bmatrix} p_1 & 0 \\ 0 & p_2 \end{bmatrix}. \quad (2.7)$$

Instead, if we allow rational (including polynomial) dependency on the scheduling variables, we can obtain the following embedding using only $p_1 = x$:

$$\ddot{x} = \frac{1}{m} (u - (k_1 + k_2 p_1^2)x - (d_1 + d_2 p_1)\dot{x}), \quad (2.8)$$

which has the following LPV-LFR representation:

$$G_{\text{rat}} : \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ z_1 \\ z_2 \\ z_3 \\ y \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ -\frac{k_1}{m} & -\frac{d_1}{m} & -1 & 1 & 1 & \frac{1}{m} \\ -\frac{k_2}{m} & 0 & 0 & 0 & 0 & 0 \\ -\frac{k_2}{m} & 0 & 1 & 0 & 0 & 0 \\ 0 & -\frac{d_2}{m} & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ w_1 \\ w_2 \\ w_3 \\ u \end{bmatrix}, \quad \Delta_{\text{rat}}(p) = \begin{bmatrix} p_1 & 0 & 0 \\ 0 & p_1 & 0 \\ 0 & 0 & p_1 \end{bmatrix}. \quad (2.9)$$

Next, we consider the system within a bounded subset of the state-space, $-1 \leq x_1 \leq 1.2$ and compute the resulting scheduling range. We find that the scheduling set $\mathbb{P}$ is defined, in the rational case as, $-1 \leq p_1 \leq 1.2$ and in the affine case additionally with $0 \leq p_2 \leq 1.44$, illustrated in Figure 2.3. Notice that,

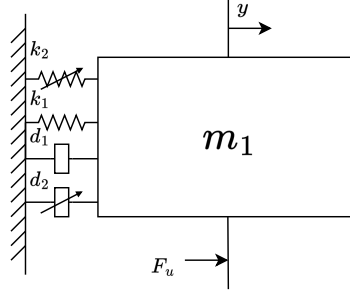


Figure 2.2: Illustration of NL-MSD used as example.

the inherent coupling between p_1 and p_2 is lost in the affine case. By sampling the scheduling set of each embedding we can determine local LTI models. Based on these models we generate the bode magnitude diagrams of the scheduling sets of both rational and affine dependency models, displayed in Figure 2.4. This figure illustrates that the set of LTI systems corresponding to the affine-dependency model is much larger than that of the rational-dependency model. This difference is caused by the overbounding of the affine model, in fact the underlying NL system can never admit the behaviour described by the difference between the sets.

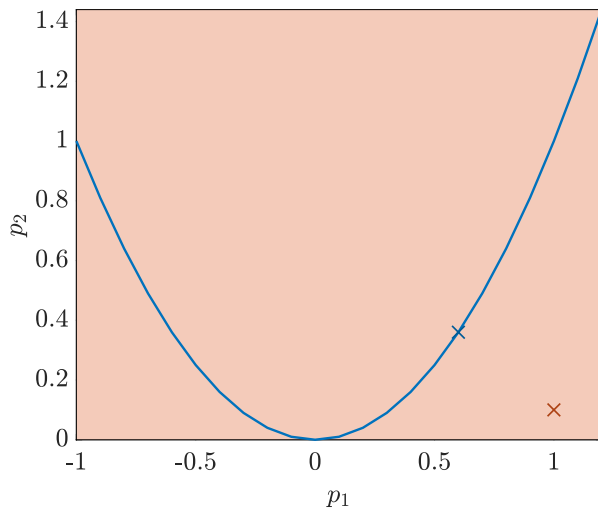


Figure 2.3: Scheduling set of the affine embedding of the NL-MSD (—) and the subset of the scheduling space occupied by the rational embedding of the NL-MSD (—)

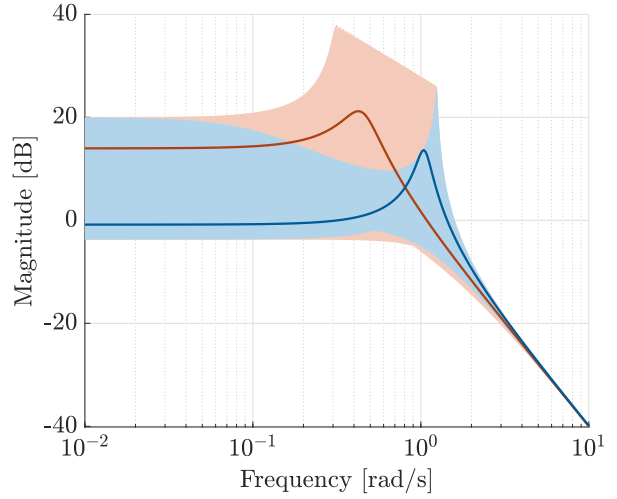


Figure 2.4: Bode magnitude plots of the LTI systems captured by the affine embedding of the NL-MSD (—) and the rational embedding of the NL-MSD (—). The lines denote sampled LTI systems as marked in Figure 2.3

2.2 | Well-posed LPV-LFR models

As we have shown in (2.4), for an LPV-LFR to be well-posed, we require $I - D_{zw}\Delta(p(t))$ to be non-singular, i.e., $\det(I - D_{zw}\Delta(p(t))) \neq 0$ for all possible realizations of the scheduling variables, i.e., $p(t) \in \mathbb{P}$. We propose a novel method for guaranteeing well-posedness of LPV-LFR models, but first we establish the assumptions and conditions that play a key role in this result.

Assumption 2.1. *The matrix function $\Delta(p(t))$ has a diagonal structure.*

Assumption 2.2. *The scheduling set $\mathbb{P}$ is contained in the closed n_p -dimensional ℓ_∞ unit ball, i.e.,*

$$\mathbb{P} \subseteq \{p : \mathbb{T} \rightarrow \mathbb{R}^{n_p} \mid \|p(t)\|_\infty \leq 1 \ \forall t \in \mathbb{T}\}. \quad (2.10)$$

Remark 2.3. *It is important to note that Assumption 2.2 is not restrictive. Given an arbitrary, well-posed, LFR $\{G, \Delta(p)\}$, under Assumption 2.1 it is trivial to retrieve an equivalent LFR defined by the pair $\{\bar{G}, \bar{\Delta}(p)\}$ which satisfies Assumption 2.2 by scaling and shifting the scheduling variables, see e.g., [23].*

Condition 2.4. *The spectral radius of D_{zw} is strictly smaller than 1. i.e.,*

$$\rho(D_{zw}) < 1. \quad (2.11)$$

These lead to the following theorem.

Theorem 2.5. *If Assumption 2.2, Assumption 2.1, and Condition 2.4 hold, the LPV-LFR model (2.1) is well-posed, i.e., $I - D_{zw}\Delta(p(t))$ is non-singular for all $p(t) \in \mathbb{P}$.*

Proof. From Assumption 2.2 and Assumption 2.1 it follows that $\rho(\Delta(p(t))) \leq 1 \ \forall p(t) \in \mathbb{P}$. Additionally, under Assumption 2.1, it holds that $\rho(D_{zw}\Delta(p(t))) \leq \rho(D_{zw})\rho(\Delta(p(t)))$. Thus, it follows that $I - D_{zw}\Delta(p(t)) \succ 0$ for all combinations of $p(t) \in \mathbb{P}$. This implies $\det(I - D_{zw}\Delta(p(t))) > 0$, $\forall p(t) \in \mathbb{P}$, satisfying the well-posedness condition.

To ensure Condition 2.4 is satisfied, we define

$$D_{zw} = e^{-N}, \quad N \succ 0. \quad (2.12)$$

As the matrix exponential maps negative definite matrices to matrices $D_{zw} \prec I$. By connecting the well-posedness condition to positive-definiteness of N , a much simpler condition is recovered, note that N is not required to be symmetric. In optimization, enforcing $N \succ 0$ through constraints could provide a method for guaranteeing well-posedness. Instead, we propose the use of direct parameterizations satisfying $N \succ 0$, similar to [24].

To guarantee Condition 2.4 through D_{zw} defined according to (2.12) with $N \succ 0$, we use the parameterization

$$N = \Psi(D_A^\top D_A + D_B - D_B^\top + \epsilon I), \quad (2.13)$$

where $\Psi = \text{diag}(e^{D_d})$ such that N is constructed with the free variables $D_A \in \mathbb{R}^{n_w \times n_w}$, $D_B \in \mathbb{R}^{n_w \times n_w}$ and $D_d \in \mathbb{R}^{n_w}$ and a fixed constant $0 < \epsilon \ll 1$. By combining a symmetric positive-semi-definite term $D_A^\top D_A$, a skew-symmetric term $D_B - D_B^\top$ and a strictly positive definite term ϵI , with positive scaling by Ψ , N is guaranteed to be strictly positive-definite by construction. The positive scaling term Ψ is shown in [24] to extend the feasible domain of N . Additionally, it aids in counteracting potential poor scaling of variables which can be introduced through the transformation (2.12).

When considering the use of LPV-LFR models in identification, the parameterization of N according to (2.13) enables unconstrained optimization without risk of obtaining ill-posed models. By substituting the direct free parameterization of D_{zw} with the transformed free variables in D_A, D_B and D_d for constructing D_{zw} , the number of parameters is increased from n_w^2 to $2n_w^2 + n_w$. However, as noted in [25], by populating only the lower triangular elements of D_A and the strictly upper triangular elements of D_B , this overparameterization is greatly reduced, now requiring only $n_w^2 + n_w$ elements, n_w variables more than populating D_{zw} directly.

2.3 | Identification Setting

For the identification of NL systems, we consider the general identification procedure proposed in [20]. Specifically, the goal is to identify *discrete-time* (DT) NL systems. We focus on DT models as data-records used in identification are in DT and evaluation of CT models is computationally more challenging. However, in theory, the methods described below can be extended for identification of CT models.

We assume DT models with innovation-type noise structure, admitting the following form:

$$\Sigma_o : \begin{cases} r_{k+1} = f_r(r_k, u_k, e_k), \\ y_k = g_r(r_k, u_k) + e_k, \end{cases} \quad (2.14)$$

where $k \in \mathbb{Z}$ is the discrete time, $r \in \mathbb{T} \subseteq \mathbb{R}^{n_r}$, $e_k \in \mathbb{R}^{n_y}$ is the realization of a white noise signal, u_k is assumed to be quasi-stationary. The state transition function $f_r : \mathbb{T} \times \mathbb{U} \times \mathbb{E} \rightarrow \mathbb{T}$ and output function $g_r : \mathbb{T} \times \mathbb{U} \rightarrow \mathbb{Y}$ are considered to be deterministic and nonlinear.

It is assumed that the dynamics (2.14) can be separated into a deterministic *plant process*, G_o , according to

$$G_o : \begin{cases} x_{k+1} = f_x(x_k, u_k), \\ y_{o,k} = g_x(x_k, u_k), \end{cases} \quad (2.15)$$

and a stochastic *noise process*, H_o , according to

$$H_o : \begin{cases} s_{k+1} = f_s(s_k, x_k, u_k, e_k), \\ v_k = g_s(s_k, x_k, u_k) + e_k, \end{cases} \quad (2.16)$$

where $s_k \in \mathbb{S} \subseteq \mathbb{R}^{n_s}$ is the state of the noise model and $v_k \in \mathbb{V} \subseteq \mathbb{R}^{n_y}$ is the output additive form of the noise process. $f_x : \mathbb{X} \times \mathbb{U} \rightarrow \mathbb{X}$ and $g_x : \mathbb{X} \times \mathbb{U} \rightarrow \mathbb{Y}$ are the state transition and output functions of the plant and $f_s : \mathbb{S} \times \mathbb{X} \times \mathbb{U} \times \mathbb{E} \rightarrow \mathbb{S}$ and $g_s : \mathbb{S} \times \mathbb{X} \times \mathbb{U} \times \mathbb{E} \rightarrow \mathbb{V}$ are the state transition and output functions of the noise representation. A full explanation of the conditions under which this separation is possible is given in [20]. The output of the deterministic plant, $y_{o,k}$, is considered to be observed under output additive noise, $y_k = y_{o,k} + v_k$, filtered by the stochastic noise process according to (2.16).

Through structural constraints on (2.16), the combined plant and noise structure is capable of representing a wide range of NL noise scenarios, including *Nonlinear AutoRegressive model with eXogenous input* (NARX), *Nonlinear Output-Error* (NOE), *Nonlinear AutoRegressive Moving Average model with eXogenous input* (NARMAX) and *Nonlinear Box-Jenkins* (NBj). This provides a systematic way of modelling nonlinear systems which is similar to methods popular for LTI systems.

Notably, by substituting $e_k = v_k - g_s(s_k, x_k, u_k)$ into the equations of (2.16), a direct characterization of the inverse noise process, H_o^{-1} can be made:

$$H_o^{-1} : \begin{cases} s_{k+1} = \bar{f}_s(s_k, x_k, u_k, v_k), \\ e_k = \bar{g}_s(s_k, x_k, u_k) + v_k, \end{cases} \quad (2.17)$$

where $\bar{f}_s(s_k, x_k, u_k, v_k) = f_s(s_k, x_k, u_k, v_k - g_s(s_k, x_k, u_k))$ and $\bar{g}_s(s_k, x_k, u_k) = -g_s(s_k, x_k, u_k)$. Allowing for direct computation of the NL one-step ahead predictor. Given in state-space representation by:

$$x_{k+1} = f_x(x_k, u_k), \quad (2.18a)$$

$$s_{k+1} = \bar{f}_s(s_k, x_k, u_k, y_k - g_x(x_k, u_k)), \quad (2.18b)$$

$$\hat{y}_k = g_x(x_k, u_k) - \bar{g}_s(s_k, x_k, u_k). \quad (2.18c)$$

2.4 | Model Structures

We aim to identify the data-generating plant G_o and noise process H_o (if present) based on a dataset $\mathcal{D}_N$ in the form of parameterized surrogate models. Specifically, we consider *self-scheduled* LPV-LFR models, requiring the joint-estimation of LPV model and scheduling map.

We parameterize the surrogate model for the plant (2.15) in an LPV-LFR form according to:

$$G_\theta : \begin{cases} \begin{bmatrix} \hat{x}_{k+1} \\ \hat{z}_k \\ \hat{y}_k \end{bmatrix} = \begin{bmatrix} A_x & B_w & B_u \\ C_z & D_{zw} & D_{zu} \\ C_y & D_{yw} & D_{yu} \end{bmatrix} \begin{bmatrix} \hat{x}_k \\ \hat{w}_k \\ u_k \end{bmatrix}, \\ \hat{w}_k = \Delta_x(\hat{p}_k) \hat{z}_k. \end{cases} \quad (2.19)$$

By eliminating the latent variables, the LPV-SS representation can be recovered:

$$\begin{bmatrix} \mathcal{A}_x(\hat{p}_k) & \mathcal{B}_x(\hat{p}_k) \\ \mathcal{C}_x(\hat{p}_k) & \mathcal{D}_x(\hat{p}_k) \end{bmatrix} = \begin{bmatrix} A_x & B_u \\ C_y & D_{yu} \end{bmatrix} + \begin{bmatrix} B_w \\ D_{yw} \end{bmatrix} \Delta_x(\hat{p}_k) (I - D_{zw} \Delta_x(\hat{p}_k))^{-1} \begin{bmatrix} C_z & D_{zu} \end{bmatrix}. \quad (2.20)$$

For the purpose of optimization, all the parameters associated with the plant are collected in the vector $\theta_G = [\text{vec}(A_x)^\top \dots \text{vec}(D_{yu})^\top]^\top$. We directly characterize the surrogate model of the inverse noise process (2.16) as:

$$H_\theta^{-1} : \begin{cases} \begin{bmatrix} \hat{s}_{k+1} \\ \hat{\zeta}_k \\ \hat{e}_k \end{bmatrix} = \begin{bmatrix} A_s & B_\omega & B_v \\ C_\zeta & D_{\zeta\omega} & 0 \\ C_e & D_{e\omega} & I \end{bmatrix} \begin{bmatrix} \hat{s}_k \\ \hat{\omega}_k \\ \hat{v}_k \end{bmatrix}, \\ \hat{\omega}_k = \Delta_s(\hat{p}_k) \hat{\zeta}_k. \end{cases} \quad (2.21)$$

where $\zeta, \omega \in \mathbb{R}^{n_\omega}$ denote the latent variables of the noise process. Ensuring the noise process retains the innovation-type representation requires setting $D_{ev} = I$, and $D_{\zeta v} = 0$ or $D_{e\omega} = 0$. In the remainder of this thesis, we populate H_θ^{-1} according to (2.21) with $D_{\zeta v} = 0$. the parameters of H_θ^{-1} are collected in the vector $\theta_H = [\text{vec}(A_s)^\top \dots \text{vec}(D_{e\omega})^\top]^\top$. Furthermore, it should be noted that the representation capability of this split model and noise process is equivalent to state-space models in innovation form, commonly used in identification of LPV models.

One benefit of using an LPV-LFR is that the inverse of an LFR is also an LFR. If the original noise process H_θ has to be recovered this is easily achieved by taking the inverse of H_θ^{-1} :

$$H_\theta : \begin{bmatrix} \hat{s}_{k+1} \\ \hat{z}_k \\ \hat{v}_k \end{bmatrix} = \begin{bmatrix} A_s - B_v D_{ev}^{-1} C_e & B_\omega - B_v D_{ev}^{-1} D_{e\omega} & B_v D_{ev}^{-1} \\ C_\zeta - D_{\zeta v} D_{ev}^{-1} C_e & D_{\zeta\omega} - D_{\zeta v} D_{ev}^{-1} D_{e\omega} & D_{\zeta v} D_{ev}^{-1} \\ -D_{ev}^{-1} C_v & -D_{ev}^{-1} D_{e\omega} & D_{ev}^{-1} \end{bmatrix} \begin{bmatrix} \hat{s}_k \\ \hat{\omega}_k \\ \hat{e}_k \end{bmatrix}. \quad (2.22)$$

Note that by eliminating the latent variables, these models can be collapsed into an LPV-SS form:

$$\begin{bmatrix} \mathcal{A}_s(p_k) & \mathcal{B}_s(p_k) \\ \mathcal{C}_s(p_k) & \mathcal{D}_s(p_k) \end{bmatrix} = \begin{bmatrix} A_s & B_v \\ C_e & I \end{bmatrix} + \begin{bmatrix} B_\omega \\ D_{e\omega} \end{bmatrix} \Delta_s(\hat{p}_k) (I - D_{\zeta\omega} \Delta_s(\hat{p}_k))^{-1} \begin{bmatrix} C_\zeta & 0 \end{bmatrix}. \quad (2.23)$$

Interestingly, under well-posed parameterization and monic noise model of H_θ^{-1} , H_θ is also guaranteed to be well-posed.

In this work, we consider the use of self-scheduled LPV models, where we expect the scheduling signal p_k to be defined by a (possibly NL) scheduling map $\psi(x_k, u_k, d_k)$, where $d_k \in \mathbb{R}^{n_d}$ is a known exogenous signal and $\psi : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \times \mathbb{R}^{n_d} \rightarrow \mathbb{R}^{n_p}$ is a function. Here d_k is included to enable full or partial external scheduling, while if $n_d = 0$, true self-scheduling is recovered.

To be able to capture the set of data-generating systems described in (2.14), the scheduling map is required to be a general function approximator. To achieve this, we propose the use of an FNN as surrogate for the scheduling map. Typically the FNN is extended with a linear bypass layer, resulting in a one-block ResNet [26]. This bypass aids in the learning of linear dependencies on x_k, u_k and d_k if present. As a baseline choice, a ResNet with two hidden layers, limited amount of neurons per layer (6-12) and hyperbolic tangent activation functions is suggested. We additionally include an output activation function, required to meet Assumption 2.2 for well-posedness, resulting, in the case of a 2-layer ResNet, in the following scheduling map:

$$\hat{p}_k := \psi_\theta(\hat{x}_k, u_k, d_k) = \sigma_3 \left(\sigma_2 \left(W_2 \sigma_1 \left(W_1 \sigma_0 \left(W_0 \begin{bmatrix} \hat{x}_k \\ u_k \\ d_k \end{bmatrix} + b_0 \right) + b_1 \right) + b_2 \right) + W_{\text{lin}} \begin{bmatrix} \hat{x}_k \\ u_k \\ d_k \end{bmatrix} \right), \quad (2.24)$$

where σ_0, σ_1 and σ_2 correspond to the activation functions of the input layer and the two hidden layers respectively. W_0, W_1 and W_2 correspond to the weighting matrices for the input and hidden layers and σ_3 represents the output activation function used to meet the well-posedness conditions. W_{lin} denotes the weighting matrix of a possible linear bypass layer. b_0, b_1 and b_2 denote the biases of the input and hidden layers. All parameters corresponding to the scheduling map are collected in the vector θ_ψ .

2.5 | Identification Method

We aim to estimate a parametric surrogate model using LFR parameterizations according to (2.19), (2.21) and given a dataset $\mathcal{D}_N$ by minimizing the criterion:

$$V_N^{\text{fit}}(\theta, \mathcal{D}_N) = \frac{1}{N} \sum_{k=0}^{N-1} \|\hat{e}_{k|\theta}\|_2^2, \quad (2.25)$$

where $e_{k|\theta}$ is the one-step-ahead prediction error defined as:

$$\hat{e}_{k|\theta} = y_k - \hat{y}_{k|\theta}, \quad (2.26)$$

which is calculated using the one-step ahead prediction equations (2.18). The parameter vector θ is comprised of the combined parameter vectors of the plant, noise model and scheduling map, i.e., $\theta = [\theta_G^\top \theta_H^\top \theta_\psi^\top]^\top$. The minimization of (2.25) matches the classical *Prediction Error Minimization* (PEM) criterion which aims to minimize the *Mean Squared Error* (MSE) of the predictions given a parameterized model with regard to observations contained in a dataset $\mathcal{D}_N$.

Additionally, it can be beneficial to extend the criterion (2.25) by adding a regularization term $V_N^{\text{reg}}(\theta)$. In particular we consider the use of ℓ_2 -regularization. Two distinct types are considered. As first option, ℓ_2 -regularization on the parameter vector θ , defined as:

$$V_N^{\text{reg}}(\theta) = \rho_\theta \|\theta\|_2, \quad (2.27)$$

where ρ_θ is a regularization weight on the parameters and $\|\theta\|_2$ denotes the ℓ_2 norm of the parameter vector θ . Secondly, if the initial conditions are trained as parameters we consider ℓ_2 -regularization on the initial conditions x_0 and z_0 . To this end, we collect the estimated initial conditions in the parameter vectors θ_{x_0} and θ_{s_0} . This leads to the following regularization term:

$$V_N^{\text{reg}}(\theta) = \rho_{x_0} \left\| \begin{bmatrix} \theta_{x_0} \\ \theta_{s_0} \end{bmatrix} \right\|_2, \quad (2.28)$$

where ρ_{x_0} is the regularization weight for the initial conditions x_0 and s_0 .

In theory, also other types of regularization can be supported, in particular the use of non-smooth regularization terms such as ℓ_1 - and group-lasso regularization are proposed in [11]. The latter is effectively used in [12] for model order selection of self-scheduled LPV-SS models with affine scheduling dependency. However, these types of regularization are not considered for the remainder of this work.

By combining the PEM criterion (2.25), regularization terms, one-step-ahead prediction equations (2.18) and the LPV-LFR model structure, the identification problem is formulated as follows:

$$\begin{aligned} \underset{\theta}{\text{argmin}} \quad & V_N^{\text{fit}}(\theta, \mathcal{D}_N) + V_N^{\text{reg}}(\theta) \\ \text{s.t.} \quad & x_0 = \theta_{x_0} \\ & s_0 = \theta_{s_0} \\ & \hat{p}_k = \psi(\hat{x}_k, u_k, d_k, \theta) \\ & \hat{x}_{k+1} = \mathcal{A}_x(\Delta_x(\hat{p}_k), \theta) \hat{x}_k + \mathcal{B}_x(\Delta_x(\hat{p}_k), \theta) u_k \\ & \hat{y}_k = \mathcal{C}_x(\Delta_x(\hat{p}_k), \theta) \hat{x}_k + \mathcal{D}_x(\Delta_x(\hat{p}_k), \theta) u_k \\ & \hat{s}_{k+1} = \mathcal{A}_s(\Delta_s(\hat{p}_k), \theta) \hat{s}_k + \mathcal{B}_s(\Delta_s(\hat{p}_k), \theta) (y_k - \hat{y}_k) \\ & \hat{e}_k = \mathcal{C}_s(\Delta_s(\hat{p}_k), \theta) \hat{s}_k + (y_k - \hat{y}_k) \end{aligned} \quad (2.29)$$

To solve this optimization problem, we leverage the optimization approach described in [11] based on JAX [27]. In this gradient-based method, local optimization is performed in two steps. First, a fixed number of iterations is taken using the Adam algorithm [28], followed by a number of iterations using the *bound-constrained limited-memory Broyden-Fletcher-Goldfarb-Shanno* (L-BFGS-B) algorithm [29]. The use of Adam, through the incorporation of *momentum*, aids in escaping local minima at the early iterations of the optimization process, providing a better point of initialization for the L-BFGS-B optimizer, which subsequently converges accurately to the final minimum of the optimization process. This two-step approach has been shown to be computationally efficient for the identification of NL-SS systems [11] as well as self-scheduled LPV-SS models [12], particularly when compared with methods using only Adam. To calculate the gradients of the objective function in (2.29) with respect to the parameter vector θ , required for the Adam and L-BFGS-B algorithms, *Automatic Differentiation* (AD) is used. We extend the optimization approach described in [11] to suit the LPV-LFR model structure described in (2.19) and (2.21). This is supported by the flexibility of the AD-based optimization approach, as changes to the cost function or model structure do not require manual recalculation of Jacobians required.

Simulation of the well-posed LPV-LFR models requires solving a linear system of equations at each timestep, this operation can be numerically and computationally challenging. JAX solves the linear system at each time step, through the use of an LU-decomposition approach. Notably, for additional computational efficiency JAX implements efficient methods to evaluate the gradient using AD over the linear solve operations. Through use of the implicit function theorem, the gradients of linear solves are defined directly at the solution, ignoring unnecessary factorization operations. This approach enables evaluation of LPV-LFR models with rational dependency without increasing evaluation time excessively.

2.6 | Parameter Initialization

When starting the optimization procedure, initial values for all elements in the parameter vector are required. By default, this is done a pseudo-random manner. The elements of the plant are initialized as follows:

$$\begin{aligned} A_x &= 0.5I, & B_w &= \mathbf{0}, & B_u &\sim \mathcal{U}(-0.1, 0.1), \\ C_z &\sim \mathcal{U}(-0.1, 0.1), & D_{zu} &\sim \mathcal{U}(-0.1, 0.1), & C_y &\sim \mathcal{U}(-0.1, 0.1), \\ D_{yw} &= \mathbf{0}, & D_{yu} &= \mathbf{0}. \end{aligned} \quad (2.30)$$

If affine LPV-LFR models are used, D_{zw} is left out of the parameter vector, fixed at a zero matrix. For well-posed LPV-LFR models, we initialize components used to construct the D_{zw} matrix as follows: the lower triangular elements of D_A^x are drawn from a uniform distribution $\mathcal{U}(0, 0.1)$, the strictly upper triangular elements of D_B^x follow the normal distribution $\mathcal{N}(1, 1)$, finally $D_d^x = \mathbf{0}$ is chosen to initialize the remaining parameter vector.

The LFR parameters of the noise model are initialized similarly to their plant equivalents with a change in the scaling of A_s and the omission of $D_{\zeta v}$ and D_{ev} as these are fixed at a zero matrix and identity matrix respectively, to ensure the noise process is monic. The remaining parameters are initialized using the following strategy:

$$\begin{aligned} A_s &= 0.1I, & B_\omega &= \mathbf{0}, & B_v &\sim \mathcal{U}(-0.1, 0.1), \\ C_\zeta &\sim \mathcal{U}(-0.1, 0.1), & C_e &= \mathbf{0}, & D_{e\omega} &= \mathbf{0}. \end{aligned} \quad (2.31)$$

As final part of the noise model initialization, the components of $D_{\zeta\omega}$ are initialized using the same procedure as used for the initialization of D_{zw} . Lastly, to initialize the parameters of the scheduling map, represented by a ResNet, uniform Xavier initialization [30] is used. Bias terms are initialized to zero. We emphasize that, in general, complete (pseudo-) random initialization following the procedure above is discouraged, unless strictly necessary. As is shown in Section 4, more sophisticated initialization approaches paired with bootstrapping methods can have significant benefit for the optimization process.

The optimization algorithms used for system identification can be sensitive to scaling. To prevent poorly scaled problems, it is customary to scale the input and output data in a dataset $\mathcal{D}_N$. Typically, data records are scaled to have an empirical standard deviation of 1. Typically, the data pre-processing step also includes shifting signals such that they are centered around zero. In our method, this is not done by default as purely LPV models cannot support constant offsets in input and output. Since the goal is

learning global LPV-models, introducing additional offsets or trimming values to account for the shift on the training data is considered undesirable.

However, for improved learning of the FNN a bias term is included for the exogenous signals that enter the scheduling map directly, u_k and d_k . This is done to aid learning of the FNN, particularly to prevent issues with vanishing gradients. The Xavier initialization method assumes zero mean input signals. To achieve this regardless, the mean of input signals in the training set $\mathcal{D}_{\text{train}}$ are saved as parameters of the scheduling map: $u_{\text{offset}} = \bar{u}_{\text{train}}$, $d_{\text{offset}} = \bar{d}_{\text{train}}$. Resulting in the scheduling map $\hat{p} := \hat{\psi}(\hat{x}_k, u_k - u_{\text{offset}}, d_k - d_{\text{offset}})$.

2.7 | Simulation Example

To illustrate feasibility of the described identification methodology we show a numerical example based on the NL-MSD system described in Equation (2.5). The NL system described in Equation (2.5) is discretized using a first order Euler method, resulting in the following DT description of the system.

$$\begin{aligned} x_{1,k+1} &= x_{1,k} + T_s x_{2,k}, \\ x_{2,k+1} &= x_{2,k} + \frac{T_s}{m} (u - k_1 x_{1,k} - k_2 x_{1,k}^3 - d_1 x_{2,k} - d_2 x_{1,k} x_{2,k}), \end{aligned} \quad (2.32)$$

where $x_1 = x$ and $x_2 = \dot{x}$ of the original continuous time description. Using the DT system we generate two datasets, one for training and one for validation. Each dataset is generated by exciting the system with a different realization of a random phase multisine signal. Each signal is $N = 10^3$ samples long and has frequency components from 0.01 Hz to 50. The input signals have amplitudes with standard deviation $\sigma_u = 5$ N. The used input signals and their resulting outputs, which are generated without noise, are displayed in Figure 2.5.

To match the example in Section 2.1.1, we identify different models. First, we aim to identify an LPV-LFR model with affine dependency with $n_p = 2$ and $\eta = 1$ (number of repetitions in the Delta block). Second, we perform the same identification on a LPV-LFR model with rational dependency using our novel well-posed parameterization, with $n_p = 1$ and $\eta = 3$. Finally a simple affine dependency-based LPV-LFR with $n_p = 1$ and $\eta = 3$ is considered. All models have $n_x = 2$, $n_s = 0$ and have a ResNet scheduling map with 2 hidden layers, 6 neurons per hidden layer and $\tanh()$ activation functions. Moreover, to reduce complexity in this example, D_{yw} and D_{zu} are forced to zero. This results in a model which, when collapsed, admits scheduling dependency only in the $\mathcal{A}$ matrix, matching the LPV embeddings described in Equations (2.6) and (2.8).

To reduce challenges resulting from bad initializations, we initialize the A_x, B_u, C_y and D_{yu} matrices to match the true values of the LPV embedding, ensuring that the initial model estimates match the LTI behaviour of the LPV embedding without the influence of scheduling variables. After initialization, each model is optimized for 1000 epochs with Adam with a learning rate of 10^{-3} and a maximum of 4000 epochs using L-BFGS.

To evaluate the performance of the identified models, we consider the *Best Fit Rate* (BFR) defined as:

$$\text{BFR} = \max \left\{ 1 - \frac{\sum_{k=0}^{N-1} \|y_k - \hat{y}_k\|_2}{\sum_{k=0}^{N-1} \|y_k - \bar{y}\|_2}, 0 \right\} \cdot 100\%, \quad (2.33)$$

where $\hat{y}_k$ is the estimated y and $\bar{y}$ is the sample mean of y . For each trained model, the BFR is calculated on the validation dataset. The affine LPV-LFR ($n_p = 2$) attains a BFR of 98.97% and has training time of

Table 2.1: Model parameters for the DT NL-MSD used in simulation.

Parameter	Symbol	Value	Unit
Sampling Time	T_s	0.01	s
Mass	m	1.0	kg
Spring	k_1	0.1	$\text{N} \cdot \text{m}^{-1}$
NL Spring Hardening	k_2	1.0	$\text{N} \cdot \text{m}^{-3}$
Damping	d_1	1.0	$\text{Ns} \cdot \text{m}^{-1}$
NL Damping	d_2	-0.8	$\text{Ns} \cdot \text{m}^{-2}$

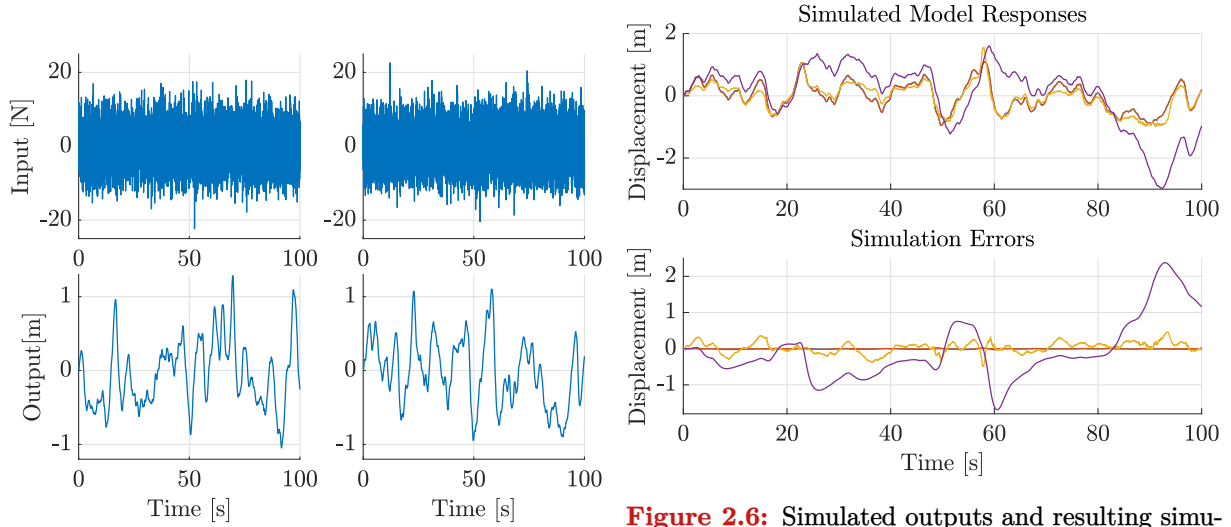


Figure 2.5: Training (left) and validation (right) datasets of the NL-MSD

Figure 2.6: Simulated outputs and resulting simulation error of the estimated models using LTI (—), simple LPV-LFR-A ($n_p = 1$) (—), LPV-LFR-A ($n_p = 2$) (—) and LPV-LFR-WP (—) model structures on the noise-free NL-MSD

147 s. The WP LPV-LFR ($n_p = 1$) attains a BFR of 97.73% in 217 s. The simple affine LPV-LFR ($n_p = 1$) attains a BFR of 65.03% in 136 s. As the data is synthesized without artificial noise, the theoretically achievable value of the BFR is 100%. The corresponding simulated trajectories on the validation data of these models are displayed in Figure 2.6. Based on these results, we draw the following conclusions: (i) Both affine dependency and well-posed rational dependency models are capable of learning a high quality surrogate model of the data-generating system under the condition that the model order is sufficiently high. No model achieves the true BFR of 100%. It is expected that this is a result of the gradient-based optimization method converging to local optima. (ii) As expected, the rational model has an increase in training time based on the increased computational complexity of evaluating this type of model. However, the approximately 60% increase in training time indicates that the method remains viable even with the added complexity of rational models and well-posed parameterizations. (iii) The difference between the simple affine and the rational LPV-LFR is significant (65% vs 98% BFR). This indicates that with $n_p = 1$ and $\eta = 3$, an affine-dependency model cannot learn (or approximate) the dynamics of data-generating system well, indeed the more complex scheduling dependency admitted by the rational models is required and does achieve this.

2.8 | Discussion

In this chapter we have presented a method for guaranteeing well-posedness of LPV-LFR models with rational scheduling dependency which is suited for unconstrained optimization. The research focus has been concentrated on the use of the novel well-posedness conditions in the joint-estimation of self-scheduled LPV models and their scheduling maps, however it must be emphasized that the proposed well-posedness conditions are not only suitable for these types of models. In fact, following suitable scaling and shifting of the scheduling variables, as discussed in Remark 2.3, these conditions can be used for traditional, externally scheduled LPV-LFR models too.

The evaluation of the matrix exponential can be computationally challenging. Because the use of this function plays a key role in our well-posed parameterization we briefly remark on its implementation in JAX. JAX implements the scaling and squaring method described in [31] in the `expm()` function. Calculation of gradients is performed through automatic-differentiation of this algorithm. A newer, more accurate, algorithm for calculating matrix exponentials is described in [32], but is currently not implemented in JAX. As an alternative operation, the Cayley transform could also be used, such as in [33], however no in-depth comparison between the performance of these transformations has been performed.

Furthermore, we have shown that a major benefit of LPV-LFR models with rational dependency is that in some cases overbounding/conservativeness can be reduced when compared to models with affine

dependency. This can provide significant advantage when used in control. In our research this potential advantage is left unresearched. Finding methods to quantify the amount of overbounding/conservativeness could help in model order and dependency selection and should be explored in the future.

3 | Multiple-Shooting-Based Identification

In this chapter we discuss and experimentally analyze various types of shooting methods in joint-estimation of LPV-LFR models and scheduling maps. First, an overview of existing research on shooting methods and their properties is given. Second, the extensions required to incorporate *Unconstrained Multiple Shooting* into the identification method proposed in Section 2 are explained. Finally, effectiveness of the unconstrained multiple shooting method in the joint-estimation of LPV-LFR models and scheduling maps is investigated by means of Monte-Carlo analysis on two identification examples.

3.1 | Introduction

In prediction error based system identification, calculation of the cost function requires evaluation of the model and dataset. In cases where (partially) simulated models are used, this requires the forward simulation over the entire length the training record. For example, to evaluate the PEM-cost of LPV-LFR models as proposed in this work, simulation of the state-trajectory is always required, as direct measurement of states is assumed unavailable.

Simulating the entire trajectory from start to finish is typically called *Single Shooting*. In modern system identification the use of single shooting presents two challenges. (i) As shown in [34], for increasing lengths of the training records the cost function becomes more irregular, particularly for unstable or slowly contracting models. As a result, the performance of gradient-based optimization methods is severely negatively affected. (ii) The recurrent evaluation of the parameterized model is not amenable to parallelization, resulting in under utilization of the capabilities of modern hardware.

Various alternative *shooting methods*, aimed at overcoming the shortcomings of single shooting, exist. In [35] *Incremental Single Shooting* (ISS) is proposed. In this method, the model is trained on an initially short, increasingly longer subset of the full training record, aiding in stability of the optimization. In *Multiple Shooting* (MS), originally used for parameter identification in [36], the full training record is instead split into multiple sub records for which identification is performed jointly. By splitting the training data into shorter sub records, large errors in simulation are prevented from building up. However, for accurate simulation, the initial states of each subrecord must now also be estimated. In [36] and [37] each timestep is treated as a different subrecord and the initial state of each subrecord is made equal to the final state of the previous subrecord. Alternatively, in [34], longer subrecords are used. Instead of constraining the first state of each subrecord to be equal to the final state of the previous subrecord, [38] uses soft constraints. This is achieved by introducing ℓ_2 -regularization on the difference of final state of the previous subrecord and the initial state of the current subrecord. In [39] it is shown that even if no constraints or regularization on initial and final states of the subrecords is used, *unconstrained multiple shooting* (UMS) still provides benefits in optimization, particularly for systems that are (locally) non-contractive or when initial model estimates are non-contractive. [40] adapts MS to (i) use overlapping subrecords to increase data-efficiency and (ii) to solve the initial state estimation problem by jointly training a NN-based state-encoder.

3.2 | Multiple Shooting-based optimization

With the goal of improving the reliability and achievable quality in the JAX-based identification introduced in Section 2 we extend the identification method to support MS. Specifically, we opt to use UMS as this is the simplest method in terms of implementation and tuning. Constrained methods are not considered, as the L-BFGS-B algorithm, which is key to the identification method, does not support the required type of constraints, only box constraints. The soft-constrained multiple shooting method could be used instead, however this adds complexity in terms of tuning of the associated regularization parameter.

To extend the identification method proposed in Section 2 to support UMS, an alternative cost function is considered. Instead of the full *prediction loss*, the *truncated prediction loss* is considered. In this method the prediction loss is evaluated on short subrecords after which the sum of the MSE of each subrecord is minimized, resulting in the following description of the cost function:

$$V_N^{\text{MS}}(\theta) = \frac{1}{TN_{\text{rec}}} \sum_{t \in \mathcal{I}} \sum_{k=0}^{T-1} \|y_{t+k} - \hat{y}_{t+k|t}\|_2^2, \quad (3.1)$$

where N_{rec} denotes the total amount of subrecords, T the length (in terms of samples) of a subrecord and $\mathcal{I}$ the set of indices of corresponding to the first index of each subrecord. In this work we only consider partitioning of N -length records into equally size, non-overlapping subrecords of length T . This results in $\mathcal{I} = \{0, T-1, 2T-1, \dots, (N_{\text{rec}}-1)T-1\}$. Each subrecord has its associated initial conditions x_t and s_t for $t \in \mathcal{I}$. These initial conditions are all considered free variables and gathered in the optimization vector θ_{x_0} , resulting in an extension of the optimization variable vector by $(N_{\text{rec}}-1)(n_x + n_s)$ variables compared to the single shooting case.

3.3 | Simulation Example - CMG

For the purpose of analyzing the effect of multiple shooting methods on the LPV-LFR identification process, we first consider the *3-Degrees of Freedom (3DOF) Control Moment Gyroscope (CMG)*. This model and its datasets are used in [10] and [20] to evaluate performance of alternative state of the art joint estimation methods for the identification of LPV models and scheduling map, allowing direct comparison of these methods to the novel method described in this paper.

3.3.1 | Description of the data-generating system

The 3-DOF CMG, as shown in Figure 3.1 consists of a disk mounted inside a blue gimbal, which in turn is mounted inside a red gimbal which is mounted inside of a silver frame. The disk, gimbal and frames are each equipped with a DC motor for actuation and an encoder to measure the angular positions (q_1, q_2, q_3 and q_4 for the disk, blue gimbal, red gimbal and silver frame respectively). The following operating mode of the CMG is considered:

- The red gimbal is locked at $q_3 = 0$.
- The blue gimbal q_2 is actuated with a motor current i_2 .
- The angular velocity of the silver frame $\dot{q}_4$ is measured and considered as output.
- The velocity of the disk $\dot{q}_1$ is controlled independently. Tracking a random multi-level-reference signal with amplitudes between 30 and 50 rad/s and dwell time of 4-8 seconds.

This resulting dynamics can, for small angles of q_2 , be interpreted as a NL-SISO model between the input i_2 and output q_4 with dynamics dependent on the external input $\dot{q}_1$ (or in the LPV case, external scheduling). For a full overview of the dynamics we refer the reader to [41].



Figure 3.1: Picture of CMG considered in simulation.

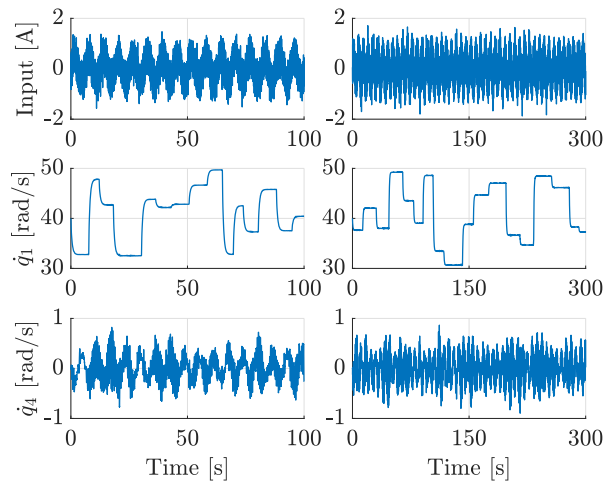


Figure 3.2: Input, exogenous scheduling signal, and output signals of the training (left) and validation (right) datasets belonging to the 3DOF CMG.

3.3.2 | Data sets

As [10] describes, the input of the blue gimbal is excited with a sinusoidal carrier wave with white noise added: $u_k := \frac{1}{2} \sin(\omega T_s k) + v_k$ where $v_k \sim \mathcal{N}(0, \sigma_v^2)$ and $\sigma_v = \frac{1}{3}$. For each dataset, the frequency of the sinusoidal carrier wave is drawn from a uniform distribution, $\omega \sim \mathcal{U}(1, 2)$. Furthermore, an OE noise structure is considered with error signal $e_k \sim \mathcal{N}(0, \sigma_e^2)$ with variance $\sigma_e^2 = 2.2 \cdot 10^{-5}$ corresponding to a SNR of 35 dB. The velocity of the disk follows the previously specified reference. The data is generated using CT simulation with RK4 integration and is subsequently down-sampled to 100 Hz. A training set consisting of 10^4 samples is created as well as a validation set of $3 \cdot 10^4$ samples. The datasets are shown in Figure 3.2.

3.3.3 | Identification Method

To enable comparison with the alternative identification methods for self-scheduled LPV models, we choose to identify models with $n_x = 5, n_p = 3$, equal to values used in previous works. We aim to identify OE LPV-LFR models (so $n_s = 0$) as the OE assumption matches the theoretical noise model of the data-generating system. Analysis of impact of multiple shooting effects is performed on LPV-LFR models with affine dependency, equivalent to LPV-SS models used [10] and [20], and LPV-LFR models under well-posed parameterization. For both model structures, $\eta = 1$ is chosen. The scheduling map is a 2-hidden layer ResNet with 6 neurons per hidden layer and $\tanh()$ activation functions.

For each dependency structure, we analyze the impact on the quality of identified models using an increasing number of subrecords. Specifically the models are trained using single shooting (1 subrecord) and unconstrained multiple shooting for 2, 4, 8 and 16 subrecords. For each combination of dependency structure and number of subrecords, Monte Carlo analysis is performed by training a 100 models based on a different random initialization using the method described in (2.30). After initialization each model is trained for 1000 epochs using Adam and 4000 epochs using L-BFGS. Initial state regularization is set to $\rho_{x_0} = 10^{-4}$ to prevent the initial state value of each subrecord to become excessively large during optimization.

3.3.4 | Results

After training, each model is evaluated by calculating the BFR on the validation dataset. A box plot of the resulting BFRs for varying number of subrecords of rational models is shown in Figure 3.3 and of affine models is shown in Figure 3.4, detailed results are displayed in Table 3.1. These plots clearly show that, on the CMG, affine models reliably converge to high BFR models, better than the rational well-posed models. We speculate that the additional complexity of WP models can have negative effects on learning, either through higher sensitivity to initialization, more complex loss landscapes or combinations of both. However, it should be noted that despite this, some of the rational models are capable of achieving BFR values close to the theoretical maximum of 98.19%.

The impact of the MS approach on rational models is directly clear, for 8 and 16 subrecords the median increases significantly, to values above 90%. This indicates that, on this dataset, UMS provides significant benefit in how reliably high quality rational models can be identified. Curiously, the number of failed simulations does not show a decrease when the amount of subrecords is increased. We speculate that these values are caused by unstable simulations on the validation data set. As the validation set is triple the length of the training set, errors during simulation can compound. This can direct trajectories of simulations to parts of the state-space on which the models are not trained, resulting in unpredictable behaviour. This is possibly worsened by the more complex scheduling dependency admitted by rational models. Why more accurate rational models are not achieved during training is unclear, this might be caused by a combination of the added complexity by the incorporation of nonzero D_{zw} terms as well as the initialization approach of its values. For affine models, the UMS approach seems to provide very slight improvement, with the median BFR consistently increasing as the number of subrecords is increased and the spread of the outliers is reduced as well.

3.4 | Simulation Example - NL-MSD

As a second point of comparison the NL-MSD as used in Section 2.7 is considered. The system description is extended to include a nonlinear noise process, allowing analysis on the performance and convergence of

Table 3.1: Table displaying the Monte Carlo results of LPV-LFR-OE models on the CMG example. Random initialization is used.

Model	N_{rec}	Type	# Fails	Median BFR [%]	Max BFR [%]	Training Time
Affine - OE	1	Sim	1	95.58%	96.37	7 min 8 s
Affine - OE	2	Sim	1	95.58%	96.76	6 min 54 s
Affine - OE	4	Sim	0	95.82%	96.55	8 min 20 s
Affine - OE	8	Sim	0	96.00%	96.74	7 min 58 s
Affine - OE	16	Sim	0	96.07	96.63	7 min 50 s
Rational - OE	1	Sim	31	35.51%	95.86	11 min 4 s
Rational - OE	2	Sim	29	37.17%	96.17	9 min 43 s
Rational - OE	4	Sim	36	27.19%	96.69	12 min 57 s
Rational - OE	8	Sim	32	92.09%	96.40	9 min 34 s
Rational - OE	16	Sim	27	93.12%	96.36	8 min 44 s

multiple shooting methods for BJ models.

3.4.1 | Data-generating system

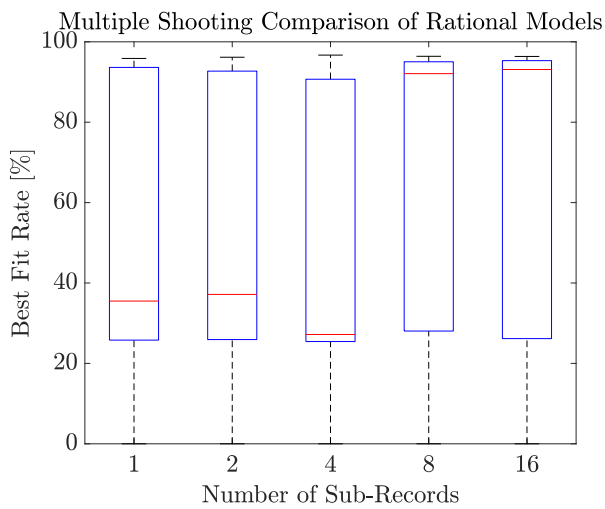
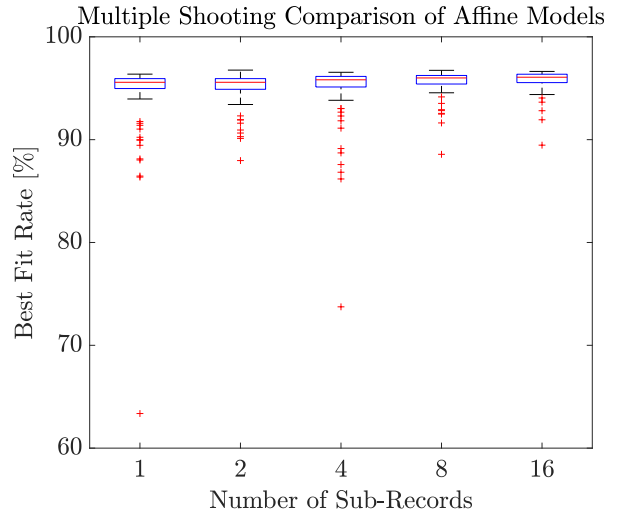
The considered data-generating system consists of a DT non-linear plant described by Equation (2.5) with parameters according to Table 2.1. Additionally we include a NL noise process of the following form:

$$H_{\text{MSD}} : \begin{cases} s_{k+1} = \left(\frac{277}{300} + \frac{2}{30} x_{1,k} \right) s_k + \left(\frac{277}{300} - 0.65 + \left(\frac{2}{30} - 0.25 \right) x_{1,k} \right) e_k, \\ v_k = s_k + e_k, \end{cases} \quad (3.2)$$

where x_1 is the position of the the plant, resulting in a position dependent noise process. This results in the total data-generating system with filtered additive noise on the output:

$$y_k = G_{\text{MSD}} u_k + H_{\text{MSD}} e_k. \quad (3.3)$$

We generate two datasets, one for training and one for validation. The input signals of the plant remain the same random phase multisine as described in Section 2.7. The noise process is excited with a Gaussian white noise signal $e_k \sim \mathcal{N}(0, \sigma_e^2)$ with $\sigma_e^2 = 1.35 \cdot 10^{-3}$. This results in datasets with a *Signal-To-Noise* (SNR) ratio of 20 dB.

**Figure 3.3:** Simulation BFR boxplots of identified LPV-LFR-OE-WP models on validation dataset of the CMG for varying N_{rec} . Random initialization is used.**Figure 3.4:** Simulation BFR boxplots of identified LPV-LFR-OE-A models on validation dataset of the CMG for varying N_{rec} . Random initialization is used.

3.4.2 | Identification Method

For identification and analysis, four types of model structures are considered, LPV-LFR-OE-A models with $n_x = 2, n_s = 0, n_p = 2, \eta = 1$, LPV-LFR-OE-WP model $n_x = 2, n_s = 0, n_p = 1, \eta = 3$, LPV-LFR-BJ-A models with $n_x = 2, n_s = 1, n_p = 2, \eta = 1$ and LPV-LFR-BJ-WP models with $n_x = 2, n_s = 1, n_p = 1, \eta = 3$. All model structures are identified with the same type of scheduling map, a 2-hidden layer ResNet with 6 neurons per hidden layer and $\tanh()$ activation functions. This enables a comparison between both rational and affine models as well as OE and BJ models on a data-generating system which admits a BJ noise structure.

Again, for each model structure the impact of changing the amount of subrecords is analyzed by performing Monte Carlo simulations. Specifically, 1, 2, 4, 8 and 16 subrecords are considered. Per model structure, 100 models are initialized using the procedure of (2.30) and (2.31) depending on the type of noise model. These are subsequently optimized according to PEM criterion Equation (2.25) with initial state regularization set to $\rho_{x_0} = 10^{-4}$.

3.4.3 | Results

First, the results of the OE affine and rational scheduling models are discussed. Boxplots of the Monte-Carlo analysis for varying number of subrecords are displayed in Figure 3.5 and Figure 3.6 for rational and affine models respectively, further details are displayed in Table 3.2. It is readily apparent that, for both types of scheduling, the optimization algorithm fails to reliably find accurate surrogate models. This could potentially be explained by two major differences between the MSD and the CMG. In the dataset for the CMG, the presence of (partial) external scheduling could aid in learning, this is not present in the MSD dataset. Secondly, MSD explores part of the state-space in which it is locally unstable. Learning this type of behaviour in simulation models can provide great challenge. It is difficult to draw conclusions on the effect of multiple shooting on the identification of rational models on this dataset. In general, with the random initialization approach, identification of high quality rational models is extremely inconsistent. However, it should be noted that for 16 subrecords, a larger grouping of outliers is shown to reach high BFRs, indicating that MS could still provide some benefit. With regard to affine models, the benefit of the MS is much more apparent, with the 16 subrecord models uniquely showing a median significantly above 0%. Comparing results with the CMG example, it seems that, for OE models, the less complex affine-dependency models more consistently converge to high quality models than the rational dependency models, indicating that the added complexity in model structure also adds complexity to the optimization process.

Next, in Table 3.3 the results of the optimized BJ models are shown. Boxplots of prediction results are shown in Figure 3.7 and Figure 3.8 for models with rational and affine dependency, respectively. Two general conclusions can directly be drawn; i) Convergence to high BFR prediction models with BJ structure is much more consistent than simulation models in OE. ii) Convergence to high BFR prediction models does *not* translate to high quality simulation models. In fact, the majority of prediction models learned using the random initialization approach perform poorly in simulation. This indicates that these models are not capable of representing dynamics of the data-generating systems, in both the affine and rational dependency case. Boxplots of the simulation BFR values are shown in Appendix A.1. The effect of MS on training rational BJ models seems to have mixed results. The single shooting model has the largest median BFR and for 4 and 8 subrecords many models perform extremely poorly. The additional number of trainable initial conditions could possibly contribute to this, requiring stronger regularization. This is particularly important with models following random initialization, as the relative contribution of regularization terms is small for poor performing models. In the affine scheduling case the benefit of MS is very clear, leading to a very tight distribution in the case of 16 subrecords.

3.5 | Discussion

In this chapter we have explored the benefits of UMS for use in joint-estimation of LPV-LFR models and scheduling maps. We have shown that, for our identification method, the use of multiple shooting can lead to a significant improvement to the reliability at which models converge to high fit rates. However, on challenging examples, such as the NL-MSD, multiple shooting alone is not always sufficient to achieve reliable convergence. We want to stress that this research only explores a small subset of what is possible with multiple shooting, and further research of additional options and configurations is strongly recom-

mended. In particular, the use of SUBNET encoders and overlapping subrecords.

One specific facet of multiple shooting not explored in the simulation examples above is performance in terms of clock-time. In cases where multiple shooting does not seemingly provide significant benefit in terms of model performance, as long as it does not significantly degrade performance, it enables parallel evaluation of the identification criterion, exploiting modern hardware more efficiently. However, the current implementation of multiple shooting in our identification method is not programmed explicitly with parallelization in mind, leaving this benefit to be a theoretical one for now. Furthermore, it should be noted that the training times noted in Monte-Carlo examples are significantly slower as a result of multiple models being trained in parallel. Training times on individual models with the same seeds are typically found to be significantly shorter. These should therefore only be used as a rough comparison between affine and rational dependency models.

Table 3.2: Table displaying the Monte Carlo results of LPV-LFR-OE models on the NL-MSD example. Random initialization is used.

Model	N_{rec}	Type	# Fails	Median BFR [%]	Max BFR [%]	Training Time
Affine - OE	1	Sim	50	0.03	88.09	3 min 40 s
Affine - OE	2	Sim	53	0	88.69	4 min 34 s
Affine - OE	4	Sim	58	0	80.09	2 min 50 s
Affine - OE	8	Sim	53	0	89.36	3 min 11 s
Affine - OE	16	Sim	38	7.98	89.54	3 min 43 s
Rational - OE	1	Sim	67	0	82.25	7 min 31 s
Rational - OE	2	Sim	67	0	77.25	7 min 32 s
Rational - OE	4	Sim	71	0	76.59	8 min 5 s
Rational - OE	8	Sim	65	0	82.16	10 min 18 s
Rational - OE	16	Sim	79	0	88.53	6 min 20 s

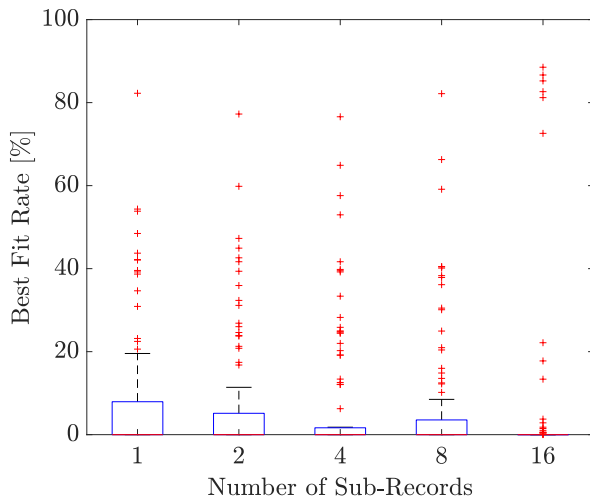


Figure 3.5: Simulation BFR boxplots of identified LPV-LFR-OE-WP models on validation dataset of the NL-MSD for varying N_{rec} . Random initialization is used.

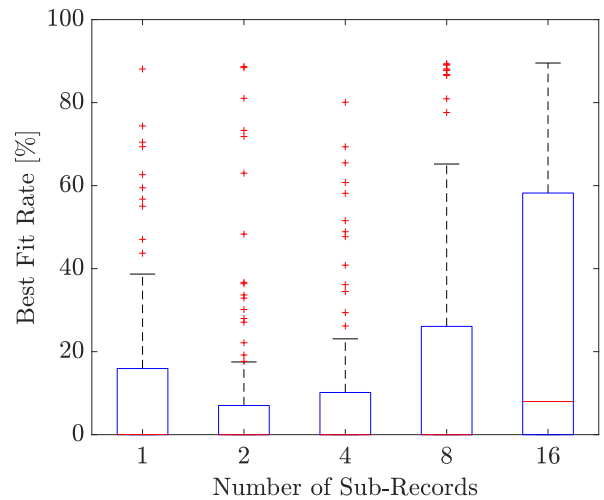


Figure 3.6: Simulation BFR boxplots of identified LPV-LFR-OE-A models on validation dataset of the NL-MSD for varying N_{rec} . Random initialization is used.

Table 3.3: Table displaying the Monte Carlo results of LPV-LFR-BJ models on the NL-MSD example. Random initialization is used.

Model	N_{rec}	Type	# Fails	Median BFR [%]	Max BFR [%]	Training Time
Affine - BJ	1	Sim	68	0	89.51	3 min 45 s
Affine - BJ	1	Pred	21	91.12	91.77	3 min 45 s
Affine - BJ	2	Sim	82	0	65.44	3 min 20 s
Affine - BJ	2	Pred	25	91.17	91.60	3 min 20 s
Affine - BJ	4	Sim	79	0	85.98	6 min 24s
Affine - BJ	4	Pred	21	91.31	91.67	6 min 24 s
Affine - BJ	8	Sim	82	0	85.94	5 min 17 s
Affine - BJ	8	Pred	8	91.32	91.67	5 min 17 s
Affine - BJ	16	Sim	72	0	89.19	5 min 27 s
Affine - BJ	16	Pred	10	91.32	91.74	5 min 27 s
Rational - BJ	1	Sim	92	0	5.88	3 min 29 s
Rational - BJ	1	Pred	39	86.20	91.51	9 min 53 s
Rational - BJ	2	Sim	92	0	74.00	9 min 42 s
Rational - BJ	2	Pred	34	82.67	91.63	9 min 42 s
Rational - BJ	4	Sim	96	0	0.64	2 min 30 s
Rational - BJ	4	Pred	61	0	91.54	9 min 21 s
Rational - BJ	8	Sim	94	0	78.10	8 min 9 s
Rational - BJ	8	Pred	72	0	91.63	8 min 9 s
Rational - BJ	16	Sim	96	0	84.90	6 min 41 s
Rational - BJ	16	Pred	58	63.07	91.67	6 min 41 s

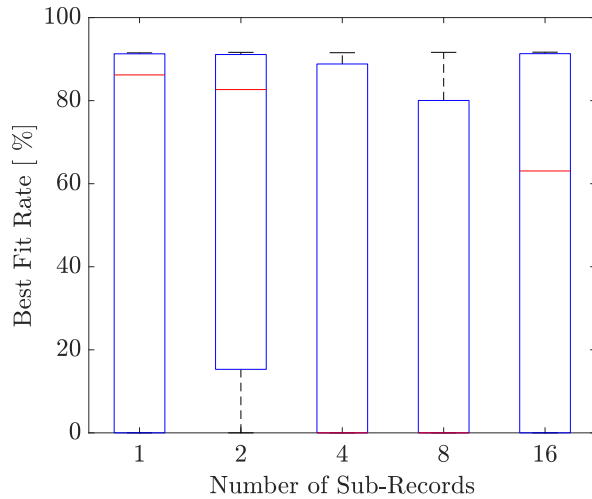


Figure 3.7: Prediction BFR boxplots of identified LPV-LFR-BJ-WP models on validation dataset of the NL-MSD for varying N_{rec} . Random initialization is used.

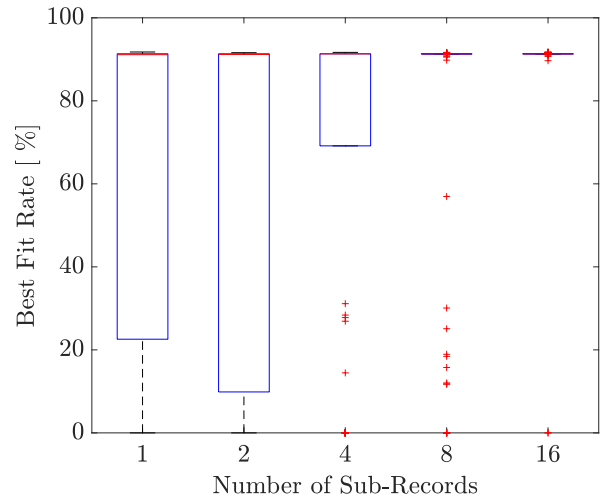


Figure 3.8: Prediction BFR boxplots of identified LPV-LFR-BJ-A models on validation dataset of the NL-MSD for varying N_{rec} . Random initialization is used.

4 | Bootstrapping approaches for the identification of LPV models

In this chapter, we introduce an initialization and bootstrapping procedure for LPV-LFR models with affine and rational dependency based on the *Best Linear Approximation*. First, we give an overview of existing initialization and bootstrapping procedures common in LPV and NL system identification. Next, we propose an LPV-LFR specific initialization and bootstrapping procedure, partially adapting the work in [42]. Finally, we investigate the performance of the new initialization and bootstrapping procedure in comparison to less sophisticated random initialization methods. This investigation is performed by means of Monte Carlo analysis on two identification examples.

4.1 | Introduction

As discussed in the previous chapter, the non-convex nature of the identification problem (2.29) can prove challenging for gradient-based optimization methods. This often leads to sub-optimal results as algorithms converge to local minima in the cost function. Because of this behaviour, particularly for complex model structures, the parameter initialization approach plays a significant role in results achieved by optimization algorithms.

To aid the initialization process, in system identification, partial prior knowledge is often incorporated in what is called a *bootstrapping* procedure, where a part of the optimization parameters is initialized with previously found values. For example, in the identification of LTI SS models, it is common to first initialize a SS-OE model based on subspace identification, optimize for simulation error and then bootstrap the SS-OE model into a combined plant and process model with BJ structure which is subsequently optimized for prediction error minimization.

In the identification of externally scheduled LPV models, initialization and bootstrapping approaches are common. Already in [16], [17], a bootstrapping approach is used in which a rational LPV-LFR model is bootstrapped from an affine LPV-LFR model which in turn is bootstrapped from an LTI model obtained using subspace identification. Despite the increase in model complexity, initialization and bootstrapping procedures for joint-estimation of self-scheduled LPV models and scheduling maps follow the same simple procedure. Both [20] and [9] bootstrap from LTI models and use random initialization for additional plant and scheduling map parameters. In [10], a random initialization without bootstrapping is employed, instead the method relies on its subspace encoder to ensure stability of the optimization process.

For the identification of more general (without LPV structure) nonlinear models, initialization approaches based on the *Best Linear Approximation* (BLA) framework [43] are used for a variety of model structures such as polynomial nonlinear state space models [44] and nonlinear-LFR models, [45], [42] of which the latter model structure is of particular importance, as it is closely related to the LPV-LFR model structure. In [42] the BLA is used to initialize the LTI parameters of an NL-LFR, moreover two important steps are taken. First, after a BLA is obtained, the model simulated and a similarity transform is performed to guarantee the sampled standard deviation of the state trajectory is 1 with mean 0. This ensures that all inputs to the neural network, used to model nonlinearities, have standard deviation 1. Which, when paired with random uniform initialization of the neural network is recognized to reduce problems with exploding and vanishing gradients in optimization. Secondly, the NL part of the NL-LFR is initialized with random or 0 values, such that the initial NL-LFR trajectory matches the BLA initialization. This initialization and scaling strategy is shown to be beneficial on two benchmarks.

4.2 | BLA-based bootstrapping procedure

We propose a three-step identification procedure for the estimation of model parameters of self-scheduled LPV-LFR-BJ models. First, an LTI model, typically the BLA, is identified. Second, using a transformed BLA estimate an LPV-LFR-OE model is initialized and jointly optimized with its scheduling map. Third, the LPV-LFR-OE model and scheduling map are used to initialize an LPV-LFR-BJ model and scheduling map.

Step 1: Best Linear Approximation

Inspired by [42], we adopt its BLA-based initialization approach and alter it to suit the LPV-LFR model

structure. First, a parametric BLA estimate in the form of an LTI-SS model is obtained using `jax-sysid`. This model has the following state-space representation:

$$G_{\text{BLA}} : \begin{bmatrix} A_{\text{BLA}} & B_{\text{BLA}} \\ C_{\text{BLA}} & D_{\text{BLA}} \end{bmatrix}. \quad (4.1)$$

The system is simulated to obtain the training dataset $\mathcal{D}_N^{\text{train}}$ with a simulated state-trajectory x^{sim} . Subsequently, an equivalent system with state-space matrices

$$\tilde{A}_{\text{BLA}} = T_x A_{\text{BLA}} T_x^{-1}, \quad \tilde{B}_{\text{BLA}} = T_x B_{\text{BLA}}, \quad \tilde{C}_{\text{BLA}} = C_{\text{BLA}} T_x^{-1}, \quad \tilde{D}_{\text{BLA}} = D_{\text{BLA}}, \quad (4.2)$$

is created. Where the transformation matrix $T_x \in \mathbb{R}^{n_x \times n_x}$ is a diagonal matrix with the inverse of the standard deviation of each sampled state trajectory, i.e.,

$$T_x = \begin{bmatrix} \sigma(x_1^{\text{sim}})^{-1} & & 0 \\ & \ddots & \\ 0 & & \sigma(x_{n_x}^{\text{sim}})^{-1} \end{bmatrix}. \quad (4.3)$$

The aim of this transformation is to ensure the state-trajectory with regard to simulation with the input record u in the training data set has unit variance.

Step 2: Initialization of LPV-LFR Output-Error models

Secondly, based on the transformed system matrices of the BLA estimate, an LPV-LFR-OE model is initialized. With the exception of D_{zw} , the remaining coefficients of the LPV-LFR model are initialized as zero or random variables:

$$\begin{aligned} A_x &= \tilde{A}_{\text{BLA}}, & B_w &= \mathbf{0}, & B_u &= \tilde{B}_{\text{BLA}}, \\ C_z &\sim \mathcal{U}(-0.1, 0.1), & D_{zu} &\sim \mathcal{U}(-0.1, 0.1), & C_y &= \tilde{C}_{\text{BLA}}, \\ D_{yw} &= \mathbf{0}, & D_{yu} &= \tilde{D}_{\text{BLA}}. \end{aligned} \quad (4.4)$$

For affine-dependency models this represents the complete initialization of the LPV-LFR model. For rational-dependency models we additionally initialize $D_{zw} = 0.5I$, which requires setting

$$D_A^x = \sqrt{-\ln(0.5)} \cdot I, \quad D_B^x = \mathbf{0}, \quad D_d^x = \mathbf{0}, \quad (4.5)$$

resulting in $D_{zw} = 0.5I$ when using the well-posed parameterization following (2.12) and (2.13). Additionally, the scheduling map with ResNet structure is initialized using uniform Xavier initialization [30] for the weight matrices, and zero initialization for all the bias vectors.

Given the initial estimate of the parameterized LPV-LFR-OE model, the optimization problem (2.29) is solved, resulting in an optimized LPV-LFR model with OE structure.

Step 3: Initialization of LPV-LFR Box-Jenkins models

Finally, we used the optimized LPV-LFR-OE model for the initialization of the LPV-LFR-BJ model. The plant model $\hat{G}_{\text{BJ}}$ and the scheduling map $\hat{\psi}_{\text{BJ}}$ are initialized with the optimal results $\hat{G}_{\text{OE}}^*$ and $\hat{\psi}_{\text{OE}}^*$ from the LPV-LFR-OE model. The LPV-LFR-BJ model is constructed by extending the model to include the inverse noise process $\hat{H}_{\text{BJ}}^{-1}$. We initialize this LFR matrix of $\hat{H}^{-1}$ according to the following strategy:

$$\begin{aligned} A_s &= 0.1I, & B_\omega &= \mathbf{0}, & B_v &\sim \mathcal{U}(-0.1, 0.1), \\ C_\zeta &\sim \mathcal{U}(-0.1, 0.1), & C_e &= \mathbf{0}, & D_{e\omega} &= \mathbf{0}, \end{aligned} \quad (4.6)$$

for affine dependency models, for rational dependency models $D_{\zeta\omega} = 0.5I$, which requires setting

$$D_A^s = \sqrt{-\ln(0.5)} \cdot I, \quad D_B^s = \mathbf{0}, \quad D_d^s = \mathbf{0}, \quad (4.7)$$

when using the well-posed parameterization of the inverse noise model.

This initialization and bootstrapping approach provides a number of benefits beyond (pseudo-)random initialization approaches of combined plant and noise models. First, the BLA-based initialization provides a good starting point in many cases, as a majority of system dynamics exhibits close to linear behaviour. Second, by leveraging the BLA estimate and performing a similarity transformation allows guaranteeing

the state-trajectory at initialization has unit variance. Together with the scaling of input-out signals discussed in Section 2, this guarantees at initialization all input signals of the scheduling map have (approximate) unit variance. This satisfies the key assumption in many of the initialization approaches used for neural networks, including the Xavier initialization employed in this work, that input signals have unit variance. Third, after the initial LTI model, each subsequent initial model estimate is initialized such that the output trajectory is unchanged in the first simulation. By setting $B_w = \mathbf{0}$ and $D_{yw} = 0$ in (4.4), the initial LPV-LFR-OE model does not show any scheduling-dependent behavior, resulting in the simulated output matching the BLA estimate. In fact, if the BLA estimate is stable this implies the initial estimate of the OE LPV-LFR model is stable too. The same is achieved for LPV-LFR-BJ models. By setting $B_w = \mathbf{0}$ and $C_e = \mathbf{0}$ in (4.6), it is ensured that the initial simulation matches the (stable) behaviour of the OE LPV-LFR estimate. This allows a gradual increase in model complexity and prevents undesired unstable initializations, which is especially important due to the recurrent nature of the models used.

4.3 | Simulation Example - CMG

To evaluate and analyze the performance of the BLA-based initialization, again a Monte Carlo approach is adopted. First, the CMG system as described in Section 3.3 is considered. The datasets displayed in Figure 3.2 are again used. The identification method and model structures considered are the same as in Section 3.3, the only distinction made is the models are now initialized according to the BLA-based initialization approach.

4.3.1 | Results

First, we compare the BFR results on validation data on models trained with single-shooting. In Figure 4.1, affine and well-posed models, using the proposed initialization and bootstrapping procedure, are compared with random initialized counterparts. Further results which also include the use of multiple shooting are shown in Table 4.1, boxplots of the full results can be found in Appendix A.2. For rational models, the novel initialization method shows significant improvement. Models with high BFR are identified much more reliably, almost all models show BFR over 90%. Only 1 model fails to converge, compared to 31 in the random initialization case. For affine models the novel initialization approach shows slightly less effect. Nonetheless, the distribution of BFRs shows tighter grouping, with the worst outlier above 90% BFR.

Observing the values in Table 4.1 shows that the initialization approach, when combined with UMS, some (slight) benefit in terms of BFR for increasing number of sub-records is present. Additionally, although less extreme than for the random initialization case, the median BFR of affine models is slightly higher than for rational models, regardless of amount of sub-records. This indicates that, even with the BLA-based initialization approach, the additional complexity of well-posed rational models impacts the optimization procedure. However, the maximum BFR achieved with rational models (96.82%) is slightly higher than that of affine models (96.67%). This could indicate that for the model order used, the underlying system can be captured more accurately with rational scheduling dependency. Notably, the best models for each scheduling dependency outperform results achieved in [20], in which a simulation BFR on validation data set of 96.50% was achieved.

4.4 | Simulation Example - MSD

As second simulation example, the same nonlinear mass-spring-damper system as in Section 3.4 is used. Using Monte-Carlo methods we analyze the impact of proposed initialization and bootstrapping procedure for affine and rational dependency models for OE and BJ noise model structures. The same data sets, model orders, and initialization parameters are used as in Section 3.4, only the initialization procedure is changed. For initializing BJ models the best out of a 100 OE models is chosen.

4.4.1 | Results

First, we analyze the impact of the BLA-based initialization approach for OE models. As shown in Figure 4.2, the initialization approach shows significant improvement in the achieved BFR as well as how reliably models achieve high BFR. For rational models a very tight grouping close to the theoretical maximum (89.97% in simulation) is achieved. Notably, no outliers are present and all models converge to

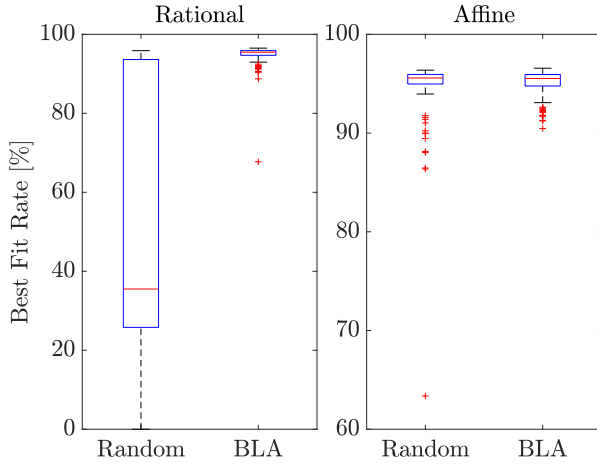


Figure 4.1: Boxplot comparison of initialization approaches for LPV-LFR-OE-WP (left) and LPV-LFR-OE-A (right) models. Simulation BFR on the validation dataset of the CMG are shown.

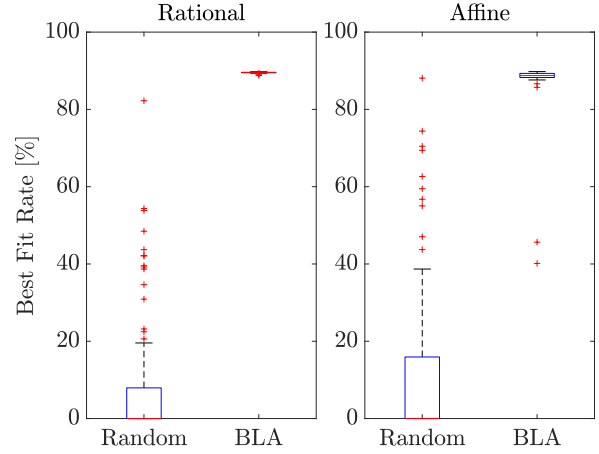


Figure 4.2: Boxplot comparison of initialization approaches for LPV-LFR-OE-WP (left) and LPV-LFR-OE-A (right) models. Simulation BFR on the validation dataset of the NL-MSD are shown.

Table 4.1: Table displaying the Monte Carlo results of LPV-LFR-OE models on the CMG example. BLA-based initialization is used.

Model	N_{rec}	Type	# Fails	Median BFR [%]	Max BFR [%]	Training Time
Affine - OE	1	Sim	0	95.53	96.57	6 min 38 s
Affine - OE	2	Sim	0	95.64	96.51	8 min 41 s
Affine - OE	4	Sim	0	95.90	96.67	8 min 54 s
Affine - OE	8	Sim	0	95.86	96.59	8 min 28 s
Affine - OE	16	Sim	0	95.99	96.65	8 min 1 s
Rational - OE	1	Sim	1	95.47	96.51	9 min 42 s
Rational - OE	2	Sim	1	95.42	96.53	12 min 21 s
Rational - OE	4	Sim	0	95.59	96.61	10 min 50 s
Rational - OE	8	Sim	0	95.82	96.82	10 min 42 s
Rational - OE	16	Sim	0	95.91	96.71	9 min 10 s

a high BFR, compared to 67 models that fail to converge in the random initialization case. For affine models, similar results are achieved, however the spread of the achieved BFR is slightly larger and includes some outliers. Potentially, this is caused by the increased complexity of the scheduling map ($n_p = 2$) required for affine dependency models.

Next, the initialization and bootstrapping approach is combined with MS. The results are shown in Table 4.2. For boxplots corresponding to these results we refer to Appendix A.2. For rational models, the addition of MS shows little effect on the median achieved, in some cases performing slightly worse ($N_{\text{rec}} = 2$) and sometimes slightly better ($N_{\text{rec}} = 4, 8$). The only major difference is present at $N_{\text{rec}} = 16$ which shows notably worse performance, possibly caused by the reduced length of each sub record. Interestingly, for affine models a different trend is shown. The spread of results steadily decreases as the number of sub records increases, with no drop at 16 sub-records. It is unclear why this difference is present.

Finally, we evaluate the performance of BJ models, initialized using the previously identified OE models. Boxplot comparison of single-shooting trained BJ models, evaluated for prediction, are shown in Figure 4.3. Both rational and affine models display a notable improvement when bootstrapped from OE models. The resulting spread of achieved BFRs is small and the median values, as shown in Table 4.3 are close to the theoretical limit on this dataset (91.80% in prediction). However, when these models are evaluated in simulation, as shown in Figure 4.4, it becomes clear that some differences between model types do exist. Despite being initialized with a high BFR OE model, the simulation performance of BJ models

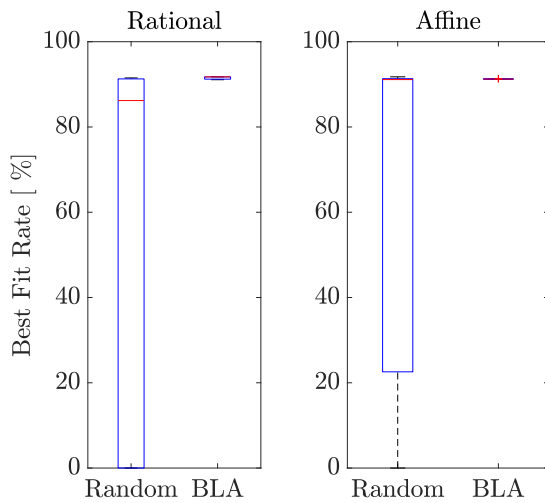
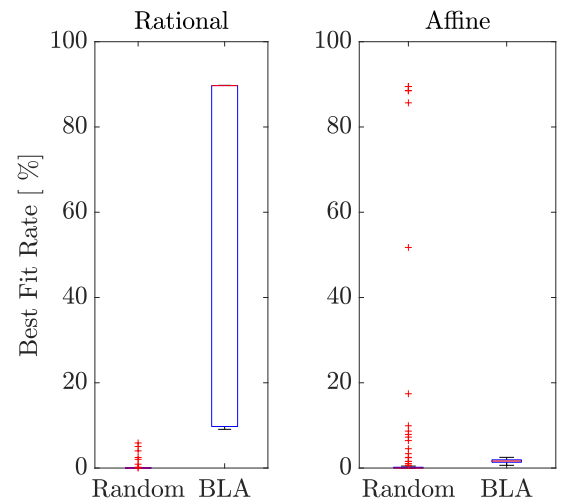
Table 4.2: Table displaying the Monte Carlo results of LPV-LFR-OE models on the NL-MSD example. BLA-based initialization is used.

Model	N_{rec}	Type	# Fails	Median BFR [%]	Max BFR [%]	Training Time
Affine - OE	1	Sim	0	88.76	89.78	3 min 51 s
Affine - OE	2	Sim	0	88.62	89.83	4 min 48 s
Affine - OE	4	Sim	0	88.45	89.73	5 min 13 s
Affine - OE	8	Sim	0	88.53	89.65	3 min 57 s
Affine - OE	16	Sim	0	88.94	89.52	4 min 20 s
Rational - OE	1	Sim	0	89.55	89.73	7 min 40 s
Rational - OE	2	Sim	0	89.52	89.67	7 min 9 s
Rational - OE	4	Sim	0	89.56	89.75	6 min 25 s
Rational - OE	8	Sim	0	89.60	89.76	6 min 16 s
Rational - OE	16	Sim	0	87.96	89.40	5 min 45 s

can deteriorate during training. For rational models, the median simulation BFR remains high, with a large spread of how models perform. For affine models, the simulation BFR is consistently below 5% indicating that, when trained using single shooting, the simulation performance is sacrificed to improve the prediction performance. This could again be explained by the affine scheduling models having a more complex scheduling map with $n_p = 2$, versus the more simple scheduling maps of rational models. Interestingly, when UMS is used instead of single-shooting, this problem is resolved, and the BJ models retain high BFR in simulation. In terms of effect on BFR in prediction, no clear trend is visible for rational models, for differing amounts of sub-records the spread of BFR values can be worse or better than single shooting, it is unclear why this happens. Affine models seem to benefit more strongly from employing multiple shooting. For a further comparison of the MS models we refer to Appendix A.2.

4.5 Discussion

In this chapter we have shown an initialization and bootstrapping approach for rational and affine LPV-LFR models. On two simulation examples we have shown this approach to significantly improve the quality of, and reliability at which, models convergence to high fit rates over less sophisticated random initialization methods. In particular for models following our well-posed parameterization, initialization and bootstrapping approach seems to provide major benefit. Despite this, a detailed theoretical understanding of the method is currently lacking and is worth exploring in the future.

**Figure 4.3:** Boxplot comparison of initialization approaches for LPV-LFR-BJ-WP (left) and LPV-LFR-BJ-A (right) models. Prediction BFR on the validation dataset of the NL-MSD are shown.**Figure 4.4:** Boxplot comparison of initialization approaches for LPV-LFR-BJ-WP (left) and LPV-LFR-BJ-A (right) models. Simulation BFR on the validation dataset of the NL-MSD are shown.

Additionally, we remark on the omission of bootstrapping from affine LPV-LFR models to rational LPV-LFR models, which is already performed in [16], [17]. During preliminary testing, it was found that on some examples, this bootstrapping step worsened performance in joint-estimation of self-scheduled LPV models and scheduling map. We speculate that, through modelling capability of the ResNet scheduling map, affine and rational models can converge to very different LPV-embeddings (or approximations) of the underlying NL dynamics. Such an affine model does not necessarily benefit from the more complex scheduling dependency of rational dependency models, and may in fact, suffer from the well-posed parameterization of rational models. However, in the case of externally scheduled models, bootstrapping from affine LPV-LFR models to rational LPV-LFR models might still be beneficial.

Table 4.3: Table displaying the Monte Carlo results of LPV-LFR-BJ models on the NL-MSD example. BLA-based initialization is used.

Model	N_{rec}	Type	# Fails	Median BFR [%]	Max BFR [%]	Training Time
Affine - BJ	1	Sim	0	1.65	2.51	1 min 16 s
Affine - BJ	1	Pred	0	91.24	91.32	1 min 10 s
Affine - BJ	2	Sim	0	89.37	89.86	2 min 10 s
Affine - BJ	2	Pred	0	91.73	91.76	5 min 44 s
Affine - BJ	4	Sim	0	89.73	89.84	5 min 40 s
Affine - BJ	4	Pred	0	91.76	91.77	5 min 40 s
Affine - BJ	8	Sim	0	88.89	89.28	4 min 54 s
Affine - BJ	8	Pred	0	91.71	91.74	4 min 54 s
Affine - BJ	16	Sim	0	88.84	89.26	4 min 29 s
Affine - BJ	16	Pred	0	91.69	91.71	4 min 29 s
Rational - BJ	1	Sim	0	89.70	89.77	3 min 45 s
Rational - BJ	1	Pred	0	91.75	91.76	9 min 58 s
Rational - BJ	2	Sim	0	89.78	89.90	9 min 42 s
Rational - BJ	2	Pred	0	91.75	91.78	8 min 53 s
Rational - BJ	4	Sim	0	89.71	89.91	2 min 30 s
Rational - BJ	4	Pred	0	91.50	91.79	8 min 10 s
Rational - BJ	8	Sim	0	89.72	89.87	8 min 9 s
Rational - BJ	8	Pred	0	91.77	91.78	7 min 12 s
Rational - BJ	16	Sim	0	88.25	88.69	6 min 41 s
Rational - BJ	16	Pred	0	91.64	91.68	6 min 27 s

5 | Model Augmentation of Linear Parameter-Varying Models

In this chapter we present a model augmentation approach for LPV-LFR models. First, background information on the topic of model augmentation is given. Subsequently in Section 5.2, the augmentation setting including augmentation options is discussed. Next, a bootstrapping approach for initializing augmented models from baseline models is presented. Finally, the augmentation approach is shown in simulation examples of two different augmentation settings.

5.1 | Introduction

As discussed in Section 1, no methods for the incorporation of prior knowledge, in terms of FPM, into the identification process of LPV models exists in literature. In the field of chemical engineering, model augmentation or *hybrid modelling*, by combining FPM with NN was introduced by [13], an extensive overview including common model structures can be found in [46]. In the fields of identification and control, the incorporation of information from FPM is still an active field of research. Common static augmentation structures include parallel augmentations [47] and series augmentations [48]. Dynamic parallel augmentations, in which the augmentations include the extension of the state vector, are used in [49] on an industrial example. Dynamic augmentations with more complex augmentation structures are used on an industrial example in [50]. In [51], a unifying interconnection approach, using LFRs to represent all commonly used types of interconnections, has been proposed. It should be noted that this approach is designed for NL-LFR models, making it not directly suitable for use with LPV-LFR models.

5.2 | Model Augmentation in LPV-LFR identification setting

In this section we present a method to incorporate prior information of a system to be identified, into the system identification setting and methodology described in Section 2. First, we assume prior information to be available in terms of an (LPV-) LFR representation. This so-called *baseline model* admits the following form:

$$G_{\theta}^b : \begin{cases} \begin{bmatrix} x_{k+1}^b \\ z_k^b \\ y_k \end{bmatrix} = \begin{bmatrix} A_x^b & B_w^b & B_u^b \\ C_z^b & D_{zw}^b & D_{zu}^b \\ C_y^b & D_{yw}^b & D_{yu}^b \end{bmatrix} \begin{bmatrix} x_k^b \\ w_k^b \\ u_k \end{bmatrix}, \\ w_k^b = \Delta_x^b(p_k^b) z_k^b. \end{cases} \quad (5.1)$$

Where the superscript $\cdot^b$ is used to denote parameters and variables associated with the baseline model. In general, this model structure admits rational dependency LPV-LFR models, including affine dependency models inherently. Additionally, by setting $n_w = 0$, LTI models can be represented.

In this thesis, only self-scheduled LPV-LFR models are considered. Baseline LPV models thus require specification of a scheduling map. Similar to (2.24), we assume the scheduling map $\psi_{\theta}^b(x_k^b, u_k, d_k)$ to be a user-defined nonlinear function of the baseline states, inputs, and possibly an exogeneous scheduling signal.

Next, to learn additional unmodelled dynamics and nonlinearities we *augment* the baseline LPV-LFR to include additional states, latent variables and/or scheduling variables. The general form of the augmented LPV-LFR is described as:

$$G_{\theta}^a : \begin{cases} \begin{bmatrix} x_{k+1}^b \\ x_{k+1}^a \\ z_k^b \\ z_k^a \\ y_k \end{bmatrix} = \begin{bmatrix} A_x^{bb} & A_x^{ba} & B_w^{bb} & B_w^{ba} & B_u^b \\ A_x^{ab} & A_x^{aa} & B_w^{ab} & B_w^{aa} & B_u^a \\ C_z^{bb} & C_z^{ba} & D_{zw}^{bb} & D_{zw}^{ba} & D_{zu}^b \\ C_z^{ab} & C_z^{aa} & D_{zw}^{ab} & D_{zw}^{aa} & D_{zu}^a \\ C_y^b & C_y^a & D_{yw}^b & D_{yw}^a & D_{yu}^b \end{bmatrix} \begin{bmatrix} x_k^b \\ x_k^a \\ w_k^b \\ w_k^a \\ u_k \end{bmatrix}, \\ \begin{bmatrix} w_k^b \\ w_k^a \end{bmatrix} = \begin{bmatrix} \Delta_x^b(p_k^b) & 0 \\ 0 & \Delta_x^a(p_k^b, p_k^a) \end{bmatrix} \begin{bmatrix} z_k^b \\ z_k^a \end{bmatrix}, \end{cases} \quad (5.2)$$

where the superscript $\cdot^a$ is used to denote parameters and variables associated with the augmented part of the model. superscripts $\cdot^{ab}$ and $\cdot^{ba}$ are used to indicate parameters that interconnect between baseline and augmentation signals. Through the augmentation of the latent variables z^a and w^a , additional parameter-varying terms can be included. Additionally, by augmenting the state vector to include x^a , dynamic augmentations can be incorporated, used to model additional states, for example caused by

higher-order dynamics not included in the baseline model. By freezing or populating subsets of the parameters in (5.2), various different augmentation structures can be recovered.

Furthermore, as shown in (5.2), the model structure admits augmentation of the scheduling variables through the inclusion of p^a , generated by the augmented scheduling map ψ^a . The choice on the type of scheduling augmentation is independent from the model structure described in (5.2). Theoretically, a wide range of interconnections between baseline and augmented scheduling maps is possible, such as parallel, series and combinations of series and parallel interconnections. In this work, we consider only the parallel augmentation case.

5.2.1 | Regularization

For LPV-LFR representations, particularly when augmented with ResNet scheduling maps, equivalent representations exhibiting the same input-output behaviour exist. This non-uniqueness can impact the interpretability of updated baseline parameters. For example, during training, dynamics initially represented by baseline parameters can shift to parameters of the augmented black-box part of the model. One method of preventing this is regularization of baseline parameters, as proposed in [52]. The identification criterion is extended to include the following additional regularization term:

$$V_{\text{reg}}(\theta) = \rho_b \|\Lambda(\theta_b - \theta_b^*)\|_2^2, \quad (5.3)$$

where ρ_b denotes the regularization coefficient and θ_b represents the set of parameters corresponding to the baseline part of the model. Furthermore, $\Lambda = \text{diag}(\theta_{\text{base}}^0)^{-1}$. Note that in the original work of [52], the model type only considers physically interpretable non-zero parameters in the baseline model. Our method differs significantly in that the baseline LPV-LFR model can contain zero (or near zero) values on certain elements the matrix. This leads to problems of infeasibility (or numerical instability) when inverted. Instead we alter the regularization method to:

$$\Lambda = \text{diag}(\lambda), \quad \lambda_i = \min \left\{ \frac{1}{|\theta_{b,i}| + \epsilon}, 10^4 \right\}, \quad (5.4)$$

where $\lambda \in \mathbb{R}^{\theta_b}$ is a vector of regularization parameters and $0 < \epsilon \ll 1$ is a small value to ensure the inversion is numerically stable, typically chosen as 10^{-16} . Furthermore the maximum scaling is clipped at 10^4 to prevent overweighting in cases of extremely small non-zero values.

5.3 | Bootstrapping procedure for augmented models

As shown in Section 4, the initialization strategy of black-box models can significantly impact attainable results in the identification of black-box LPV-LFR models. As this remains true for augmented LPV-LFR models, we propose an initialization method based on the same principles as discussed in the previous chapter. Additionally, we detail some steps important for parameter initialization unique to the model augmentation setting.

First, we assume a description of the baseline model G_θ^b and its scheduling map ψ_θ^b is known, as specified in Section 5.2. Next, as the optimization method relies on scaling the input and output signals, the model has to be scaled accordingly. To this end, we collect the scaling of each input and output signal in diagonal matrices T_u and T_y respectively. Furthermore, the same state scaling as proposed in Section 4.2 can be used, now based on simulation of the baseline model, instead of the BLA initialization. Together, these scaling procedures result in the scaled baseline parameters:

$$\begin{aligned} \tilde{A}_x^b &= T_x A_x^b T_x^{-1}, & \tilde{B}_w^b &= T_x B_w^b, & \tilde{B}_u^b &= T_x B_u^b T_u^{-1}, \\ \tilde{C}_z^b &= C_z^b T_x^{-1}, & \tilde{D}_{zw}^b &= D_{zw}^b, & \tilde{D}_{zu}^b &= D_{zu}^b T_u^{-1}, \\ \tilde{C}_y^b &= T_y^{-1} C_y^b, & \tilde{D}_{yw}^b &= T_y^{-1} D_{yw}^b, & \tilde{D}_{yu}^b &= T_y^{-1} D_{yu}^b T_u^{-1}. \end{aligned} \quad (5.5)$$

Additionally, the scheduling map of the baseline model is altered to handle the scaled inputs:

$$\tilde{\psi}^b(\tilde{u}, \tilde{x}, \tilde{d}) = \psi^b(T_u^{-1}\tilde{u}, T_x^{-1}\tilde{x}, T_d^{-1}\tilde{d}). \quad (5.6)$$

If the aim is to augment the baseline model to a rational-dependency model with well-posed parametrization, an additional step has to be taken. The well-posed parametrization requires Assumption 2.2 to hold, however, given an arbitrary LPV embedding, this is not necessarily true. Instead, we manipulate the

baseline scheduling map such that the resulting scheduling variables satisfy the required well-posedness conditions. To achieve this, the output of scheduling map is scaled and possibly trimmed with the inclusion of a saturation function. The output of the scheduling map is multiplied with a diagonal matrix T_p where the diagonal elements are chosen such that $|p_i^b| \leq 1$ for $i = 1, \dots, n_p$. If the outputs of the scheduling map are bounded, this condition on the scheduling trajectory can be guaranteed for arbitrary inputs. Alternatively, if the functions describing the scheduling map are unbounded for arbitrary inputs, scaling can be chosen such that the condition is met for bounded inputs corresponding to the expected operating range of the system. To prevent undesired ill-posedness during optimization, a saturation function can be employed to guarantee well-posedness given the scaling of the scheduling variables, defined as

$$\tilde{p}^b = \text{sat} \left(T_p \tilde{\psi}(\tilde{x}, \tilde{u}, \tilde{d}) \right), \quad (5.7)$$

with element-wise saturation function with minimum -1 and maximum 1 . To retain equivalence (within the bounds of the saturation function) with the original baseline model, this requires inverse scaling on B_w^b and D_{yw}^b , resulting in

$$\tilde{B}_w^b = T_x B_w^b T_p^{-1}, \quad \tilde{D}_{yw}^b = T_y^{-1} D_{yw}^b T_p^{-1}. \quad (5.8)$$

The dimensions of T_p change from $\mathbb{R}^{n_p} \times \mathbb{R}^{n_p}$ in (5.7) to $\mathbb{R}^{n_w} \times \mathbb{R}^{n_w}$ in (5.8). For ease of notation we denote both cases the same way, with the latter case containing repeated values of scaling parameters corresponding to the repetitions of scheduling variables in Δ_x^b .

In the following step, the baseline model is augmented and the additional model parameters are initialized. If only the scheduling dependency is augmented, the baseline model (5.1) is extended by including $B_w^{ba}, C_z^{ab}, D_{zu}^a$ and D_{yw}^a . These parameters are initialized as

$$B_w^{ba} = \mathbf{0}, \quad C_z^{ab} \sim \mathcal{U}(-0.1, 0.1), \quad D_{zu}^a \sim \mathcal{U}(-0.1, 0.1), \quad D_{yw}^a = \mathbf{0}. \quad (5.9)$$

If the baseline model is augmented to rational scheduling dependency with well-posed parametrization we additionally initialize the terms $D_{zw}^{bb}, D_{zw}^{ba}, D_{zw}^{ab}, D_{zw}^{aa}$. These terms are treated together for the purpose of the well-posed parameterization, such that

$$D_{zw} = \begin{bmatrix} D_{zw}^{bb} & D_{zw}^{ba} \\ D_{zw}^{ab} & D_{zw}^{aa} \end{bmatrix}. \quad (5.10)$$

To construct D_{zw} , the well-posed parameterization according to (2.12) and (2.13) is used with

$$D_A^x = 5I, \quad D_B^x = \mathbf{0}, \quad D_d^x = \mathbf{0}. \quad (5.11)$$

This results in D_{zw} which is numerically close to 0, matching closely the behaviour of the affinely-scheduled baseline model, despite using the well-posed parameterization. This initialization approach ensures that after augmentation the simulated trajectories match that of the baseline model, or in case of the rational augmentation, nearly match it.

Alternatively, if only dynamic augmentation is performed, the baseline model (5.1) is extended to include $A_x^{ba}, A_x^{ab}, A_x^{aa}, B_w^{ab}, B_u^a, C_z^{ba}$ and C_y^a . Here $A^{aa} = 0.5I$, $B_u^a \sim \mathcal{U}(-0.1, 0.1)$ and all other augmented terms are set to zero matrices of appropriate size. Finally, if both scheduling and dynamic augmentation is performed, the terms B_w^{aa} and C_z^{aa} are additionally included. We initialize $B_w^{aa} = \mathbf{0}$ and $C_z^{aa} \sim \mathcal{U}(-0.1, 0.1)$.

5.4 | Simulation Examples

To display feasibility and effectiveness of the proposed model augmentation procedure for LPV-LFR models, we perform static and dynamic augmentation on simulated examples. Two examples are considered, based on the NL-MSD from Section 2.1.1.

5.4.1 | Static Augmentation Example

As first example we consider the NL-MSD, we generate two new data-sets using the NL-MSD with parameters Table 2.1 with a reduced sampling time, changing to $T_s = 0.1$ s. One dataset is generated for training and one for validation. Each dataset is generated by exciting the system with a different realization of a random phase multisine signal. Each signal is $N = 6 \cdot 10^3$ samples long and has frequency

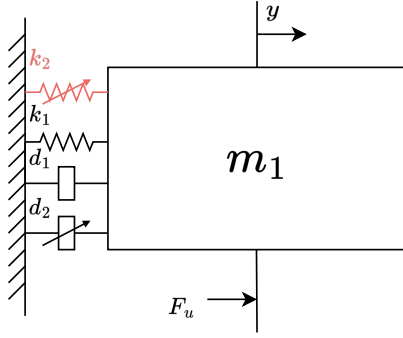


Figure 5.1: Illustration of single mass NL-MSD used in static augmentation example. Parameters used for the baseline model in black. Parameters to be learned in model augmentation in red.

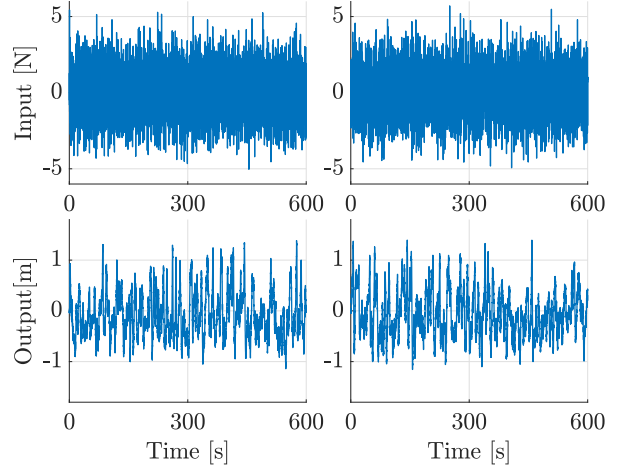


Figure 5.2: Training (left) and validation (right) datasets of the single mass NL-MSD used in the static augmentation example.

components from $\frac{1}{600}$ Hz to 50 Hz. The input signals have amplitudes with standard deviation $\sigma_u = 2$ N. The simulated outputs are considered with OE noise structure, with $e_k \sim \mathcal{N}(0, \sigma_e^2)$ and variance $\sigma_e^2 = 2.25 \cdot 10^{-3}$, resulting in an SNR of approximately 20 dB. The used input signals and their resulting outputs are depicted in Figure 5.2.

To identify this system we consider a baseline model based on LPV embeddings of part of the NL-MSD. Specifically, we consider the embedding as displayed in Figure 5.1 where linear spring, linear damper and the non-linear damping coefficient in the NL-MSD are known. The initialization of the baseline model follows the initialization approach described in Section 5.3, including saturation on the scheduling map. Next, we perform two types of augmentation. First, we augment the baseline model with affine scheduling dependency to include an additional scheduling variable, generated by a ResNet scheduling map. Secondly, we augment the baseline model to include well-posed rational scheduling dependency, and augment the delta block to include additional repetitions of the baseline scheduling signal. After initialization each model is optimized for 2000 epochs with Adam and a learning rate of 10^{-3} and a maximum of 8000 epochs using L-BFGS. Additionally, we utilize UMS with 4 sub records. Both models are trained twice, once without regularization and once with baseline regularization according to (5.3), with $\rho_b = 10^{-3}$.

As a point of comparison, black-box LPV-LFR models are also trained. The LPV-LFR model with affine scheduling dependency with $n_x = 2$, $n_p = 2$ and $\eta = 1$ is identified. A rational dependency model is also trained with $n_x = 2$, $n_p = 1$ and $\eta = 3$. Both models are trained with the same optimization parameters as the augmented models.

The results after training are displayed in Table 5.1, trajectories of the augmented models are shown in Figure 5.5. It shows that both the affine-dependency and rational-dependency augmentations are capable of reaching the theoretical maximum BFR, even with regularization. Some results show BFR higher than the data-generating system on the validation dataset, this is due to the random nature of the added noise, on other datasets these models perform slightly worse than the data-generating system.

Furthermore, to evaluate the effectiveness of the regularization method we compared the difference between the baseline A_x^b and the matrix after optimization A_x^* in terms of the 2-norm, i.e.,

$$\|A_\Delta\|_2 = \|A_x^b - A_x^*\|_2. \quad (5.12)$$

This gives an indication of the effectiveness of the baseline regularization, as in this example the augmented models do not require a change in A_x^b to capture the full data-generating system. In the case of rational scheduling dependency we see that using baseline regularization decreases to $\|A_\Delta\|_2 = 0.004$ from $\|A_\Delta\|_2 = 6.32$ for the unregularized case. For affine-dependency augmentations this is reduced to $\|A_\Delta\|_2 = 0.028$ from $\|A_\Delta\|_2 = 0.24$. This shows that in both cases the use of baseline regularization reduces the change in baseline parameters without worsening the achievable BFR.

Table 5.1: Table displaying the results of augmented models on the single mass NL-MSD example. Values of black-box models are best out of 10 runs.

Model	Type	Max BFR [%]	Training Time
Data-gen.	Sim	89.59	-
Affine - OE (black-box)	Sim	89.62	197 s
Rational - OE (black-box)	Sim	89.59	297 s
Augmented - Affine	Sim	89.34	70 s
Augmented - Affine (regularized)	Sim	89.61	232 s
Augmented - Rational	Sim	89.62	283 s
Augmented - Rational (regularized)	Sim	89.62	292 s

5.4.2 | Dynamic Augmentation Example

For the second example we consider an altered version of the NL-MSD, as displayed in Figure 5.3. In the new example the NL-MSD is extended from a 1-mass system to a 2-mass system. The second mass m_2 is connected to the first mass m_1 by a linear spring k_3 and linear damper d_3 . The corresponding parameters of the 2-mass NL-MSD are displayed in Table 5.2. Again, one dataset is generated for training and one for validation. Each dataset is generated by exciting the system with a different realization of a random phase multisine signal. Each signal is $N = 6 \cdot 10^3$ samples long and has frequency components from $\frac{1}{600}$ Hz to 50. The input signals have amplitudes with standard deviation $\sigma_u = 2$ N. The simulated outputs are considered with OE noise structure, with $e_k \sim \mathcal{N}(0, \sigma_e^2)$ and variance $\sigma_e^2 = 2.25 \cdot 10^{-3}$, resulting in an SNR of approximately 20 dB. The used input signals and their resulting outputs are displayed in Figure 5.4.

The same baseline model of the single mass NL-MSD, as in the static augmentation example is considered. Now, on top of the static augmentations for models with affine and rational dependency, we additionally augment the models to include 2 augmented states to a total of $n_x = 4$. This results in a combination of static and dynamic augmentation. After initialization each model is optimized for 2000 epochs with Adam and a learning rate of 10^{-3} and a maximum of 8000 epochs using L-BFGS. Additionally, we utilize UMS with 4 sub records. For comparison, black-box models with affine and rational scheduling dependency are trained with $n_x = 4$, according to the same optimization parameters. Both models are trained twice, once without regularization and once with baseline regularization according to (5.3), with $\rho_b = 10^{-3}$.

The performance of the resulting models is displayed in Table 5.3, trajectories of the augmented models

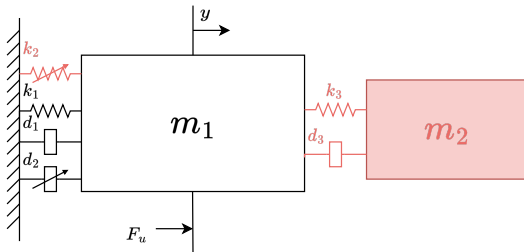


Figure 5.3: Illustration of 2-mass NL-MSD used in the dynamic augmentation example. Parameters used for the baseline model in black. Parameters to be learned in model augmentation in red.

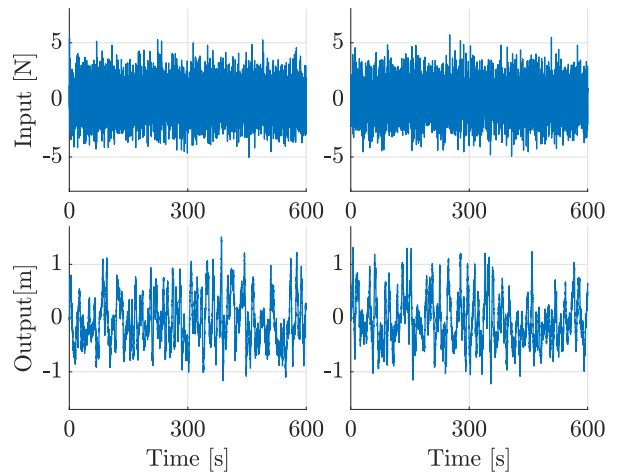


Figure 5.4: Training (left) and validation (right) datasets of the 2-mass NL-MSD used in the dynamic augmentation example.

Table 5.2: Model parameters for the 2-mass NL-MSD used in the augmentation example.

Parameter	Symbol	Value	Unit
Sampling Time	T_s	0.1	s
Mass 1	m_1	1.0	kg
Mass 2	m_2	0.5	kg
Spring 1	k_1	0.1	$\text{N} \cdot \text{m}^{-1}$
NL Spring Hardening	k_2	1.0	$\text{N} \cdot \text{m}^{-3}$
Spring 2	k_3	1.0	$\text{N} \cdot \text{m}^{-1}$
Damping 1	d_1	1.0	$\text{Ns} \cdot \text{m}^{-1}$
NL Damping	d_2	-0.8	$\text{Ns} \cdot \text{m}^{-2}$
Damping 2	d_3	0.15	$\text{Ns} \cdot \text{m}^{-1}$

are shown in Figure 5.6. It shows that, also in the case of combined static and dynamic augmentation the models achieve BFR values close to or at the theoretically achievable maximum. The black-box models are also capable of reaching this limit, however required multiple runs with different initializations to achieve it, unlike the augmented models. This indicates that initialization based on partial knowledge of the underlying system can effectively be used to aid the optimization process.

Additionally, this example shows that using baseline regularization does not have to negatively impact the achievable BFR when capturing the data-generating system requires changes in, for example, the A_x matrix such that they no longer match the baseline values. This indicates that using the baseline-regularization method is suitable (but may require tuning) in cases where the baseline parameters do not fully match the system to be identified.

5.5 | Discussion

In this chapter we introduced a model augmentation framework for the identification of LPV-LFR models with affine and rational scheduling dependency. We note however, that this is only the first step of research on this topic, and further work is strongly recommended. In particular, the examples shown have only been performed with baseline models that have partial but exact parameters of the data-generating systems. An important next step would be to investigate performance when only approximate model parameters are known. Additionally, only scheduling maps with parallel augmentation structure have been used in this chapter, although series or combined series and parallel scheduling maps might provide benefit in certain use cases.

Furthermore, in this chapter we assume baseline models in terms of LPV embeddings to be known. Deriving an LPV embedding based on a NL FPM is not always trivial, and manual derivation of LPV embeddings can be labour intensive. However, automated (exact) embeddings methods do exist and are still actively being researched, for an overview we refer the reader to [53] and the sources therein.

Table 5.3: Table displaying the results of augmented models on the 2-mass NL-MSD example. Values of black-box models are best out of 10 runs.

Model	Type	Max BFR [%]	Training Time
Data-gen.	Sim	89.40	-
Affine - OE (black-box)	Sim	89.40	261 s
Rational - OE (black-box)	Sim	89.41	343 s
Augmented - Affine	Sim	89.39	248 s
Augmented - Affine (regularized)	Sim	89.26	267 s
Augmented - Rational	Sim	89.40	318 s
Augmented - Rational (regularized)	Sim	89.40	332 s

Moreover, currently only augmentation from affine-dependency baseline models has been shown. Even if rational-dependency models are available, there is currently no method to reliably convert these models to well-posed parameterizations.

Finally, the model augmentation framework above has only been tested with OE noise model structures. In the future we recommend further research for integrating this with other types of noise models, such as shown in Section 2.

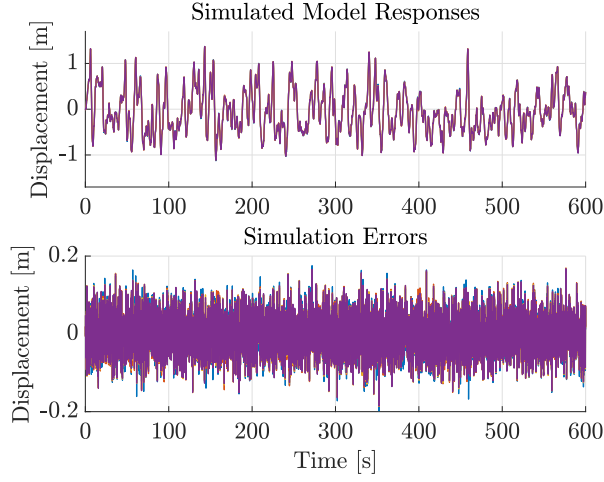


Figure 5.5: Simulated outputs and resulting simulation error of the estimated models using augmented LPV-LFR-A (—), augmented affine-dependency LPV-LFR-A (regularized) (—), augmented LPV-LFR-WP (—) and augmented LPV-LFR-WP (regularized) (—) models on the single mass NL-MSD.

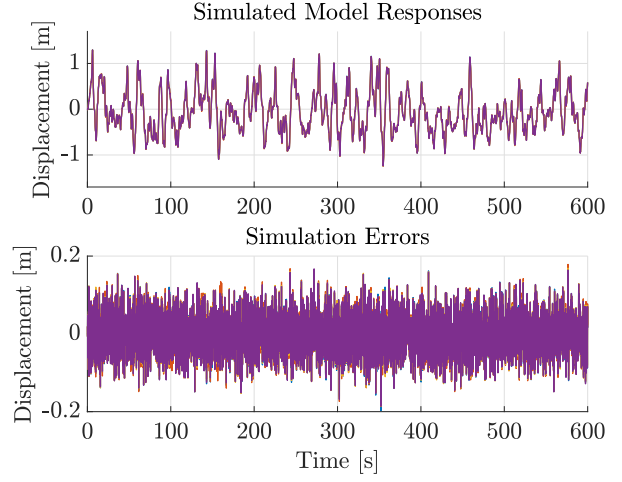


Figure 5.6: Simulated outputs and resulting simulation error of the estimated models using augmented LPV-LFR-A (—), augmented affine-dependency LPV-LFR-A (regularized) (—), augmented LPV-LFR-WP (—) and augmented LPV-LFR-WP (regularized) (—) models on the 2-mass NL-MSD.

6 | Conclusions & Recommendations

In this thesis, a novel method for guaranteeing well-posedness of LPV-LFR models has been presented. We have shown that by using a novel parameterization method, this technique is effectively incorporated in system identification methods for surrogate modelling of nonlinear systems. This approach, and the results obtained based on the method contribute to the development of modern computationally efficient nonlinear system identification methods. This chapter summarizes the most important results obtained in answering the research questions posed in Section 1 and gives recommendations for future research on this subject.

6.1 | Conclusions

In the start of this thesis, three research questions were posed with the aim of improving joint-estimation methods for identification of LPV models and scheduling maps. The first question asked how well-posedness of LPV models with LFR structure and rational scheduling dependency can be guaranteed in identification. The results of Section 2 show how, by specific parameterization, LPV-LFR models can be constrained to always be well-posed. Subsequently, this section showed how the well-posed parameterization is incorporated into a system identification approach for joint-estimation of well-posed LPV-LFR models and scheduling maps, capable of accurately modelling nonlinear models under a wide range of complex noise scenarios. Moreover, this identification approach has been shown to be computationally efficient when compared to alternative deep-learning based methods.

The second research question asked how reliable convergence to accurate surrogate models using the above-mentioned methods can be achieved. Section 3 partially answered this question by showing that, incorporating multiple shooting in the identification process, improves reliability in two Monte Carlo studies. Furthermore, Section 4 introduced a novel initialization approach for LPV-LFR models based on BLA initializations, which improves the reliability of convergence to accurate surrogate models in two Monte Carlo studies.

Finally, the third research question asked how prior knowledge of systems can be incorporated into the identification process. In Section 5 this question was answered by the introduction of a model augmentation approach for joint-estimation of LPV-LFR models and scheduling maps. This approach is shown to be effective in two simulation examples, illustrating how the versatility of the LFR model structure can be used to incorporate prior system knowledge.

6.2 | Recommendations

In answering the research questions posed in this thesis, some questions have been left open and potential avenues of future research have been found. Below we detail the most important recommendations.

First, in Section 2, we argue that LPV-LFR models with rational scheduling dependency can, in some cases, reduce overbounding/conservatism over models with affine scheduling dependency. However, currently no investigation has been performed to compare the conservatism of identified LPV-LFR models with affine and rational scheduling dependency using the well-posed parameterization. In the future, we recommend further investigation of this, by using these models in control or finding methods to quantify the amount of overbounding/conservatism.

Secondly, in Section 3, we have explored only a small subset of what is possible with multiple shooting methods. As previously discussed, methods based on SUBNET encoders, such as [10], have shown great performance on a wide range of NL system identification examples. Based on this, we strongly recommend to incorporate SUBNET encoders into the identification process described in this thesis, to evaluate its effects on performance and reliability.

Thirdly, the BLA-based initialization and bootstrapping approach proposed in Section 4 has been shown to significantly improve the reliability at which accurate models can be identified. However, a clear understanding of why this method, particularly the initialization procedure, works so well is still lacking.

As such, we recommend further research to gain a theoretical understanding of this procedure.

Finally, in Section 5.3 we have shown that incorporating prior knowledge into the identification process is possible for LPV-LFR models, however only two examples have been considered. In the future we recommend further research to explore the wide range of augmentation options possible within the LPV-LFR setting. Especially when using approximate baseline models, which are not currently considered. Moreover, the lack of systematic methods to generate baseline models with well-posed parameterizations based on LPV embeddings with rational scheduling dependency limits its use to baseline models with affine dependency. Therefore, we strongly recommend further research into methods to convert LPV models with rational scheduling dependency to well-posed parameterizations.

7 | References

- [1] J. Mohammadpour and C. W. Scherer, Eds., *Control of Linear Parameter Varying Systems with Applications*. Boston, MA: Springer US, 2012.
- [2] A. Hjartarson, P. Seiler, and A. Packard, “LPVTools: A Toolbox for Modeling, Analysis, and Synthesis of Parameter Varying Control Systems,” in *Proc. of the 1st IFAC Workshop on Linear Parameter Varying Systems*, Grenoble, France, 2015, pp. 139–145.
- [3] P. den Boef, P. B. Cox, and R. Tóth, “LPVcore: MATLAB toolbox for LPV modelling, identification and control,” in *Proc. of 19th IFAC Symposium on System Identification*. Elsevier, 2021.
- [4] R. Toth, H. Hjalmarsson, and C. R. Rojas, “Order and structural dependence selection of LPV-ARX models revisited,” in *Proc. of the 51st IEEE Conference on Decision and Control*, Maui, HI, USA, 2012, pp. 6271–6276.
- [5] P. B. Cox, “Towards efficient identification of linear parameter-varying state-space models,” Ph.D. dissertation, Eindhoven University of Technology, 2018.
- [6] J.-W. Van Wingerden and M. Verhaegen, “Subspace identification of Bilinear and LPV systems for open- and closed-loop data,” *Automatica*, vol. 45, no. 2, pp. 372–381, 2009.
- [7] P. B. Cox and R. Tóth, “Linear parameter-varying subspace identification: A unified framework,” *Automatica*, vol. 123, p. 109296, 2021.
- [8] D. Masti and A. Bemporad, “Learning nonlinear state-space models using autoencoders,” *Automatica*, vol. 129, p. 109666, 2021.
- [9] A. Rehmer and A. Kroll, “On affine quasi-LPV System Identification with unknown state-scheduling using (deep) Recurrent Neural Networks,” in *Proc. of the 26th International Conference on System Theory, Control and Computing*, Sinaia, Romania, 2022, pp. 446–451.
- [10] C. Verhoek, G. I. Beintema, S. Haesaert, M. Schoukens, and R. Toth, “Deep-Learning-Based Identification of LPV Models for Nonlinear Systems,” in *Proc. of the 61st IEEE Conference on Decision and Control*, Cancun, Mexico, 2022, pp. 3274–3280.
- [11] A. Bemporad, “An L-BFGS-B approach for linear and nonlinear system identification under ℓ_1 and group-lasso regularization,” *IEEE Transactions on Automatic Control*, 2025.
- [12] R. Drenth, “Efficient Gradient-Based Learning of Linear Parameter-Varying Models,” 2024, Internship report, Control Systems Group, Eindhoven University of Technology.
- [13] D. C. Psychogios and L. H. Ungar, “A hybrid neural network-first principles approach to process modeling,” *AIChE Journal*, vol. 38, no. 10, pp. 1499–1511, 1992.
- [14] C. Hoffmann and H. Werner, “Complexity of Implementation and Synthesis in Linear Parameter-Varying Control,” in *Proc. 19th IFAC World Congr.*, 2014.
- [15] M. Nemani, R. Ravikanth, and B. Bamieh, “Identification of linear parametrically varying systems,” in *Proc. of the 34th IEEE Conference on Decision and Control*, New Orleans, LA, USA, 1995, pp. 2990–2995.
- [16] L. Lee and K. Poolla, “Identification of linear parameter-varying systems via LFTs,” in *Proc. of 35th IEEE Conference on Decision and Control*, Kobe, Japan, 1996, pp. 1545–1550.
- [17] L. H. Lee and K. Poolla, “Identification of Linear Parameter-Varying Systems Using Nonlinear Programming,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 121, no. 1, pp. 71–78, 1999.
- [18] Y. Cheng and M. Sznaier, “Identification of LPV systems with LFT parametric dependence via convex optimization,” in *Proc. of the 54th IEEE Conference on Decision and Control*, Osaka, 2015, pp. 1459–1464.

- [19] M. Mejari, D. Piga, R. Toth, and A. Bemporad, “Kernelized Identification of Linear Parameter-Varying Models with Linear Fractional Representation,” in *Proc. of the 18th European Control Conference*, Naples, Italy, 2019, pp. 337–342.
- [20] A. Bemporad and R. Tóth, “Efficient identification of linear, parameter-varying, and nonlinear systems with noise models,” *arXiv preprint arXiv:2504.11982*, 2025.
- [21] K. Zhou, J. Doyle, and K. Glover, “Robust and optimal control,” *Control Engineering Practice*, vol. 4, no. 8, pp. 1189–1190, 1996.
- [22] A. Kwiatkowski and H. Werner, “PCA-Based Parameter Set Mappings for LPV Models With Fewer Parameters and Less Overbounding,” *IEEE Transactions on Control Systems Technology*, vol. 16, no. 4, pp. 781–788, 2008.
- [23] C. Scherer and S. Weiland, “Linear matrix inequalities in control,” 2015. [Online]. Available: <https://www.imng.uni-stuttgart.de/mst/files/LectureNotes.pdf>
- [24] M. Revay, R. Wang, and I. R. Manchester, “Lipschitz Bounded Equilibrium Networks,” *arXiv preprint arXiv:2010.01732*, 2020.
- [25] E. Winston and J. Z. Kolter, “Monotone operator equilibrium networks,” *Advances in neural information processing systems*, vol. 33, pp. 10 718–10 728, 2020.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [27] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, “JAX: composable transformations of Python+NumPy programs,” 2018. [Online]. Available: <http://github.com/jax-ml/jax>
- [28] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *arXiv preprint arXiv:1412.6980*, 2017.
- [29] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” *Mathematical Programming*, vol. 45, no. 1-3, pp. 503–528, 1989.
- [30] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. of the 13th International Conference on Artificial Intelligence and Statistics*. JMLR Workshop and Conference Proceedings, 2010, pp. 249–256.
- [31] N. J. Higham, “The Scaling and Squaring Method for the Matrix Exponential Revisited,” *SIAM Journal on Matrix Analysis and Applications*, vol. 26, no. 4, pp. 1179–1193, 2005.
- [32] A. H. Al-Mohy and N. J. Higham, “A New Scaling and Squaring Algorithm for the Matrix Exponential,” *SIAM Journal on Matrix Analysis and Applications*, vol. 31, no. 3, pp. 970–989, 2010.
- [33] C. Verhoek, R. Wang, and R. Tóth, “Learning stable and robust linear parameter-varying state-space models,” in *Proc. of the 62nd IEEE Conference on Decision and Control*, 2023, pp. 1348–1353.
- [34] A. H. Ribeiro, K. Tiels, J. Umenberger, T. B. Schön, and L. A. Aguirre, “On the smoothness of nonlinear system identification,” *Automatica*, vol. 121, p. 109158, 2020.
- [35] C. Michalik, R. Hannemann, and W. Marquardt, “Incremental single shooting—A robust method for the estimation of parameters in dynamical systems,” *Computers & Chemical Engineering*, vol. 33, no. 7, pp. 1298–1305, 2009.
- [36] H. Bock, “Recent Advances in Parameteridentification Techniques for ODE,” in *Numerical Treatment of Inverse Problems in Differential and Integral Equations: Proceedings of an International Workshop*, Bock, P. Deuflhard, and E. Hairer, Eds. Birkhäuser, 1983, pp. 95–113.
- [37] A. Van Mulders, J. Schoukens, M. Volckaert, and M. Diehl, “Two nonlinear optimization methods for black box identification compared,” *Automatica*, vol. 46, no. 10, pp. 1675–1681, 2010.
- [38] M. Forgione and D. Piga, “Continuous-time system identification with neural networks: Model structures and fitting criteria,” *European Journal of Control*, vol. 59, pp. 69–81, 2021.

- [39] J. Decuyper, M. Runacres, J. Schoukens, and K. Tiels, “Tuning nonlinear state-space models using unconstrained multiple shooting,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 334–340, 2020.
- [40] G. I. Beintema, M. Schoukens, and R. Tóth, “Deep subspace encoders for nonlinear system identification,” *Automatica*, vol. 156, p. 111210, 2023.
- [41] T. Bloemers and R. Toth, “Equations of Motion of a Control Moment Gyroscope,” Eindhoven University of Technology, Tech. Rep., 2019.
- [42] M. Schoukens and R. Tóth, “On the Initialization of Nonlinear LFR Model Identification with the Best Linear Approximation,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 310–315, 2020.
- [43] R. Pintelon and J. Schoukens, *System Identification: A Frequency Domain Approach*, 2nd ed., ser. MATLAB Examples. IEEE Press, 2012.
- [44] J. Paduart, L. Lauwers, J. Swevers, K. Smolders, J. Schoukens, and R. Pintelon, “Identification of nonlinear systems using Polynomial Nonlinear State Space models,” *Automatica*, vol. 46, no. 4, pp. 647–656, 2010.
- [45] L. Vanbeylen, “Nonlinear LFR Block-Oriented Model: Potential Benefits and Improved, User-Friendly Identification Method,” *IEEE Transactions on Instrumentation and Measurement*, vol. 62, no. 12, pp. 3374–3383, 2013.
- [46] A. M. Schweidtmann, D. Zhang, and M. Von Stosch, “A review and perspective on hybrid modeling methodologies,” *Digital Chemical Engineering*, vol. 10, p. 100136, 2024.
- [47] B. Sun, C. Yang, Y. Wang, W. Gui, I. Craig, and L. Olivier, “A comprehensive hybrid first principles/machine learning modeling framework for complex industrial processes,” *Journal of Process Control*, vol. 86, pp. 30–43, 2020.
- [48] R.-S. Götte and J. Timmermann, “Composed physics- and data-driven system identification for non-autonomous systems in control engineering,” in *Proc. of the 3rd International Conference on Artificial Intelligence, Robotics and Control*, 2022, pp. 67–76.
- [49] B. M. Kessels, A. Subrahmanian Moosath, R. H. Fey, and N. van de Wouw, “AI-based state extension and augmentation for nonlinear dynamical first principles models,” *Nonlinear Dynamics*, pp. 1–29, 2025.
- [50] A. Retzler, R. Tóth, M. Schoukens, G. I. Beintema, J. Weigand, J.-P. Noël, Z. Kollár, and J. Swevers, “Learning-based augmentation of physics-based models: An industrial robot use case,” *Data-Centric Engineering*, vol. 5, p. e12, 2024.
- [51] J. H. Hoekstra, C. Verhoek, R. Tóth, and M. Schoukens, “Learning-based model augmentation with LFRs,” *European Journal of Control*, p. 101304, Jul. 2025.
- [52] M. Bolderman, “Physics-guided neural networks for high-precision mechatronics,” Ph.D. dissertation, Eindhoven University of Technology, 2024.
- [53] E. J. Olucha, P. J. W. Koelewijn, A. Das, and R. Toth, “Automated Linear Parameter-Varying Modeling of Nonlinear Systems: A Global Embedding Approach,” in *Proc. of the 6th IFAC LPVS Workshop*, 2025.

A | Appendix A

This appendix shows additional figures based on the monte-carlo analysis performed in Section 3 and Section 4.

A.1 | Figures associated with Chapter 3

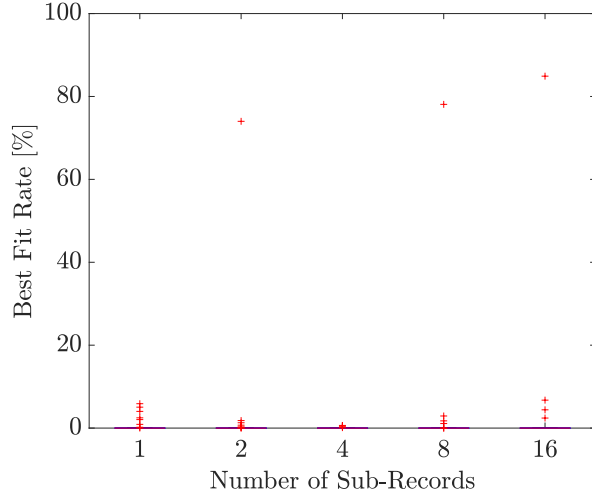


Figure A.1: Simulation BFR boxplots of identified LPV-LFR-BJ-WP models on validation dataset of the NL-MSD for varying N_{rec} . Random initialization is used.

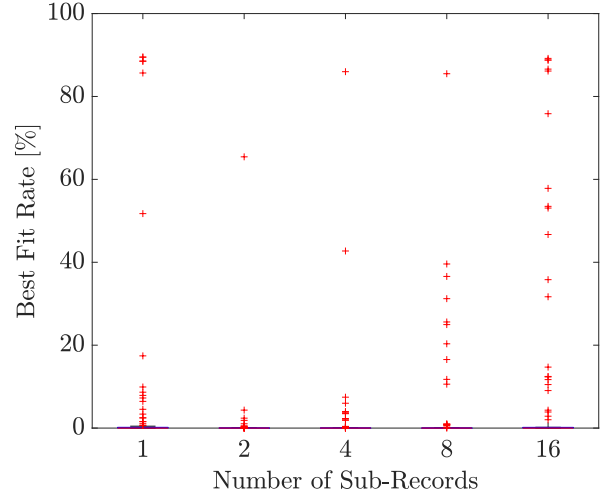


Figure A.2: Simulation BFR boxplots of identified LPV-LFR-BJ-A models on validation dataset of the NL-MSD for varying N_{rec} . Random initialization is used.

A.2 | Figures associated with Chapter 4

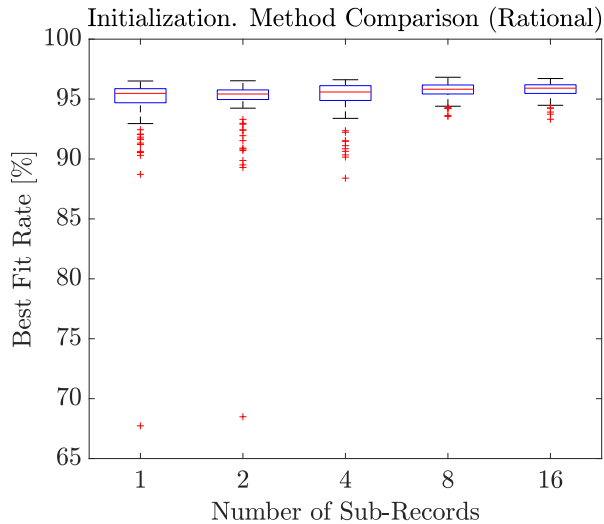


Figure A.3: Simulation BFR boxplots of identified LPV-LFR-OE-WP models on validation dataset of the CMG for varying N_{rec} . BLA-based initialization is used.

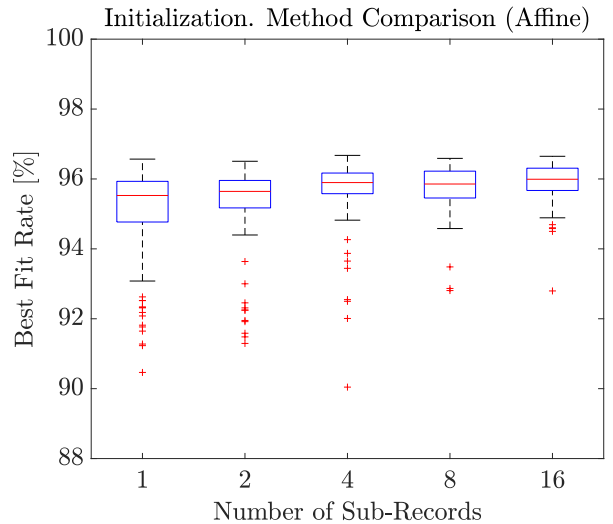


Figure A.4: Simulation BFR boxplots of identified LPV-LFR-OE-A models on validation dataset of the CMG for varying N_{rec} . BLA-based initialization is used.

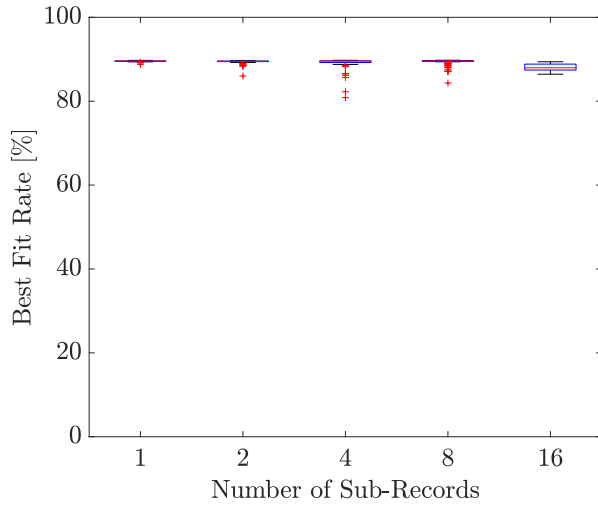


Figure A.5: Simulation BFR boxplots of identified LPV-LFR-OE-WP models on validation dataset of the NL-MSD for varying N_{rec} . BLA-based initialization is used.

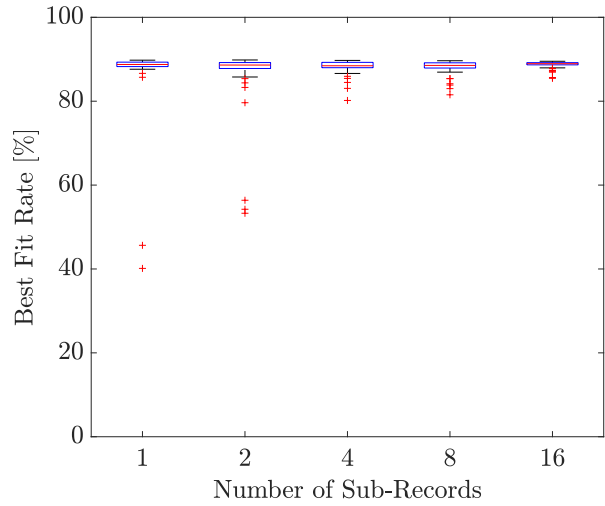


Figure A.6: Simulation BFR boxplots of identified LPV-LFR-OE-A models on validation dataset of the NL-MSD for varying N_{rec} . BLA-based initialization is used.

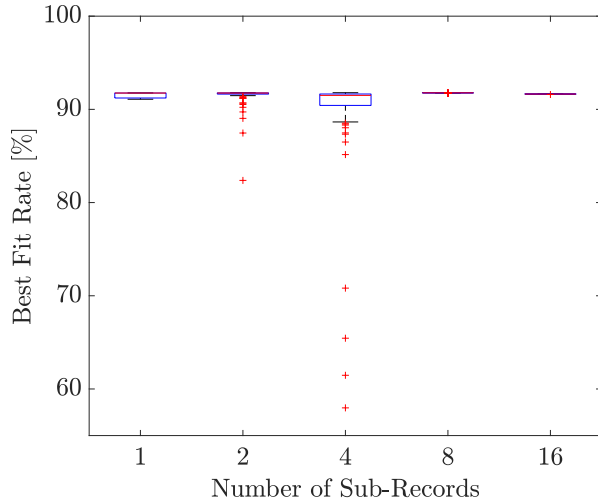


Figure A.7: Prediction BFR boxplots of identified LPV-LFR-BJ-WP models on validation dataset of the NL-MSD for varying N_{rec} . BLA-based initialization is used.

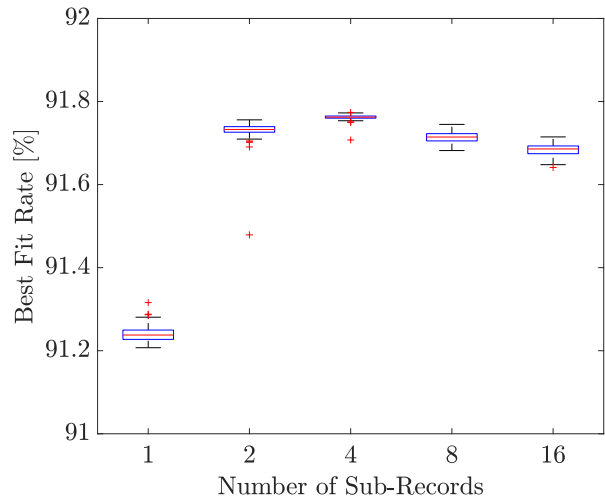


Figure A.8: Prediction BFR boxplots of identified LPV-LFR-BJ-A models on validation dataset of the NL-MSD for varying N_{rec} . BLA-based initialization is used.

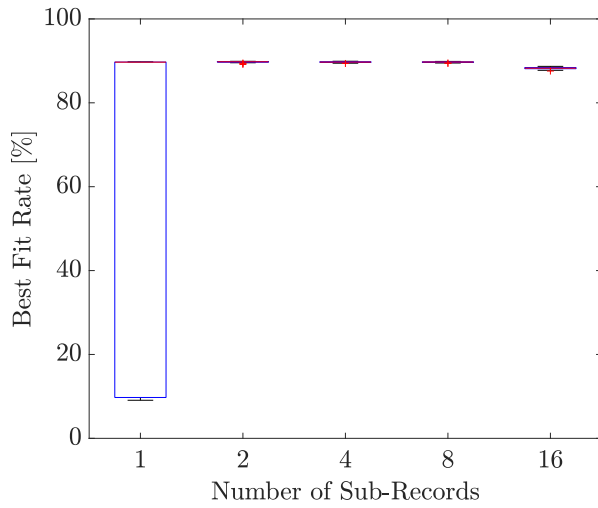


Figure A.9: Simulation BFR boxplots of identified LPV-LFR-BJ-WP models on validation dataset of the NL-MSD for varying N_{rec} . BLA-based initialization is used.

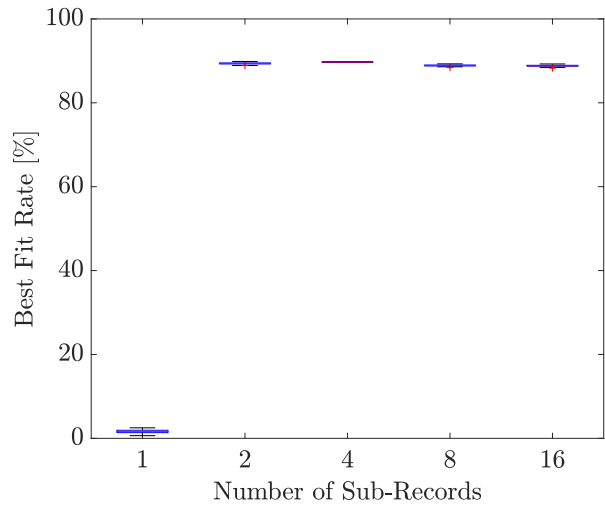


Figure A.10: Simulation BFR boxplots of identified LPV-LFR-BJ-A models on validation dataset of the NL-MSD for varying N_{rec} . BLA-based initialization is used.